Oracle® Retail
Xunit Implementation Guide

Release 25.0
G34801-01
July 2025

Oracle Retail Xunit Implementation Guide, Release 25.0

G34801-01

Copyright © 2025, Oracle and/or its affiliates.

Primary Author: Gerlinde Rust

Contributing Authors:

Contributors:

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

Contents

Send Us Your Comments

Preface

Audience i
Documentation Accessibility i
Customer Support i
Review Patch Documentation ii
Improved Process for Oracle Retail Documentation Corrections ii
Oracle Retail Documentation on the Oracle Help Center (docs.oracle.com) ii
Conventions ii

1 About Xunit

Benefits of Automated Testing 1
Xunit Advantages 1
Xunit Processing 2
Data Types 2

2 Enabling and Configuring Xunit

Enable Xunit 1
- Enabling Recording 1

Xunit Configuration 1
- system.properties 1
  - Including and Excluding Tags — Configuration 3
- log4j2-custom.xml 3
  - Action Logging 4

Mock Responders for Xstore POS 5

3 Creating and Configuring Test Cases

Recording Test Cases 1
- Action Logging 3

Copyright © 2025, Oracle and/or its affiliates.

Enable Action Logging 3
Disable Action Logging 3
Xunit Configuration 3
Parameter 4
Referencing a Parameter 4
DataPool 4
DataPool Queries 5
File Data Pool 6
TestStep 6
Using Conditions 7
Form 8
Field 8
ListSelect 8
Assertion 9
Basic Assertion 9
Prompt Assertion 10
Custom Assertion 10
Writing Strong Assertions 11
Verifying Log File Creation 11
TestSequence 12
SequenceRef 12
TestCase 13
Verifying Transaction Completion 14
TestSuite 14
Run Test 15
Excetpion 15
Disabling an Exception 16
AssertType 16
Base AssertTypes 17
Special Notations 17
!{...} Parameter Reference 18
@ Evaluation Method Chain 18
% Data Pool Accessor 19
Spring Configurations 20
xunit-beans.xml 20
Field Handlers 20
spring.xml 20
Select Methods 21
Skip Types 21

Copyright © 2025, Oracle and/or its affiliates.

4 Running Tests

Xunit Console 1
Tabs 1
Running Test Cases 2
- Messages 3
- Progress 3
- Failures 4
- Incidents 4
- Selecting Tags 4
- Xunit Setting 5

HTML Error Report 5

5 Xunit Tips and Best Practices

Daily Testing 1
File Organization 1
- Base 1
- Customer 1
- XML Schema 2

Performance Tips 2

6 Data Pools

Item 1
- Standard-Use Items 1
- Item Dimensions 3
- Item UPC 3
- Attached Item 3
- Coupon Deal Items 4
- VAT Item 4
- Restricted Item 5
- Non-Merchandise Item 5

Customer 6
House Account 6
House Account New Users 7
House Account Users 8
Transaction 9
Time Clock 9
Customer Item Account 10
Discount 11
Inventory Document 12

Copyright © 2025, Oracle and/or its affiliates.

Inventory Count 13
Inventory Valid Destination 13
Retail Location 14
Tax Exemption 14
Employee 15
Employee Borrow 16
Employee Task 17
Employee Message 18
Code Value 18
Reason Code 19
Work Code 19
Security Group 20
Sales Goals 20
Voucher 21
Raincheck 21
Customer Entry 22
Shift 22
Time Off 23
Stock Ledger 23
Serialized Stock Ledger 24
Sold Serial Numbers 24
Order 25

7 Evaluation Methods

8 Global Exceptions

Copyright © 2025, Oracle and/or its affiliates.

Send Us Your Comments

Oracle® Retail Xunit Implementation Guide, Release 25.0

Oracle welcomes customers' comments and suggestions on the quality and usefulness of this document.

Your feedback is important, and helps us to best meet your needs as a user of our products. For example:

- Are the implementation steps correct and complete?
- Did you understand the context of the procedures?
- Did you find any errors in the information?
- Does the structure of the information help you with your tasks?
- Do you need different information or graphics? If so, where, and in what format?
- Are the examples correct? Do you need more examples?

If you find any errors or have any other suggestions for improvement, then please tell us your name, the name of the company who has licensed our products, the title and part number of the documentation and the chapter, section, and page number (if available).

> **Note**
>
> Before sending us your comments, you might like to check that you have the latest version of the document and if any concerns are already addressed. To do this, access the Online Documentation available on the Oracle Technology Network Web site. It contains the most current Documentation Library plus all documents revised or released recently.

Send your comments to us using the electronic mail address: retail-doc_us@oracle.com

Please give your name, address, electronic mail address, and telephone number (optional).

If you need assistance with Oracle software, then please contact your support representative or Oracle Support Services.

If you require training or instruction in using Oracle software, then please contact your Oracle local office and inquire about our Oracle University offerings. A list of Oracle offices is available on our Web site at http://www.oracle.com.

Copyright © 2025, Oracle and/or its affiliates.

Preface

This Implementation Guide provides critical information about the processing and operating details of Xunit, including the following:

- System configuration settings
- Technical architecture
- Functional integration dataflow across the enterprise

Audience

This guide is for:

- Systems administration and operations personnel
- Systems analysts
- Integrators and implementers
- Business analysts who need information about Xunit processes and interfaces

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at http://www.oracle.com/pls/topic/lookup?ctx=acc&id=docacc.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info or visit http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs if you are hearing impaired.

Customer Support

To contact Oracle Customer Support, access My Oracle Support at the following URL:

https://support.oracle.com

When contacting Customer Support, please provide the following:

- Product version and program/module name
- Functional and technical description of the problem (include business impact)
- Detailed step-by-step instructions to re-create
- Exact error message received
- Screen shots of each step you take

Copyright © 2025, Oracle and/or its affiliates.

Review Patch Documentation

When you install the application for the first time, you install either a base release (for example, 25.0.0) or a later patch release (for example, 25.0.1). If you are installing the base release or additional patches, read the documentation for all releases that have occurred since the base release before you begin installation. Documentation for patch releases can contain critical information related to the base release, as well as information about code changes since the base release.

Improved Process for Oracle Retail Documentation Corrections

To more quickly address critical corrections to Oracle Retail documentation content, Oracle Retail documentation may be republished whenever a critical correction is needed. For critical corrections, the republication of an Oracle Retail document may at times not be attached to a numbered software release; instead, the Oracle Retail document will simply be replaced on the Oracle Help Center (docs.oracle.com) Web site, or, in the case of Data Models, to the applicable My Oracle Support Documentation container where they reside.

This process will prevent delays in making critical corrections available to customers. For the customer, it means that before you begin installation, you must verify that you have the most recent version of the Oracle Retail documentation set. Oracle Retail documentation is available on the Oracle Help Center (docs.oracle.com) at the following URL:

https://docs.oracle.com/en/industries/retail/index.html

An updated version of the applicable Oracle Retail document is indicated by Oracle part number, as well as print date (month and year). An updated version uses the same part number, with a higher-numbered suffix. For example, part number E123456-02 is an updated version of a document with part number E123456-01.

If a more recent version of a document is available, that version supersedes all previous versions.

Oracle Retail Documentation on the Oracle Help Center (docs.oracle.com)

Oracle Retail product documentation is available on the following web site:

https://docs.oracle.com/en/industries/retail/index.html

(Data Model documents can be obtained through My Oracle Support.)

Conventions

The following text conventions are used in this document:

Convention	Meaning
boldface	Boldface type indicates graphical user interface elements associated with an action, or terms defined in text or the glossary.
italic	Italic type indicates book titles, emphasis, or placeholder variables for which you supply particular values.

Copyright © 2025, Oracle and/or its affiliates.

Convention	Meaning
monospace	Monospace type indicates commands within a paragraph, URLs, code in examples, text that appears on the screen, or text that you enter.

Copyright © 2025, Oracle and/or its affiliates.

1
About Xunit

Xunit is an automation engine used to perform end user testing of the Xstore POS application without requiring user interaction. Test scripts contained in XML configuration files are loaded into the test harness, which then pushes "events" to Xstore POS, simulating user input. Test results are determined by the validity of test events based on the current system state and data assertions evaluated against the Xstore POS database, memory, and file system.

Benefits of Automated Testing

Automated software testing has the following benefits:

- **Efficiency** - Xunit can execute in a matter of seconds a regression suite that would take hours to key in manually. Quality assurance (QA) analysts spend more time analyzing results and less time typing in data.
- **Accuracy** - Actions and data assertions performed by an automation engine reduce the possibility of human error when testing the application.
- **Consistency** - Automated suites help ensure that identical test scenarios are executed from one release to the next, regardless of the QA analyst testing the system. Automation can assist in reproducing issues between separate lab environments.

Xunit Advantages

Being directly embedded within the Xstore PUS application, Xunit holds several advantages over traditional, third-party automation packages:

- Lightweight - Xunit requires minimal runtime overhead or system resources.
- Stable and fast - Test events are fired the instant the application is ready for input; running on the main application thread mitigates the risk of Xunit falling out sync with Xstore PUS during execution.
- Internationalized - Test scripts process consistently regardless of the display language of the application.
- Data validation - Xunit provides the lowest possible level of data validation because it has direct access to Xstore PUS's memory, database, and file system.
- Script maintenance - Test cases are factored into libraries of reusable components. When a change is made to the application, a single corresponding change to the test script library can accommodate all test cases relevant to the Xstore PUS functional area.
- Cost - Xunit is currently provided as part of the core Xstore PUS deliverable; no additional product or license needs to be purchased.
- Performance/load testing - Built-in capabilities to run test cases in high iteration counts, dynamically use randomized data from the Xstore PUS database, and validate system performance make Xunit the ideal utility for large-scale transaction volume and load testing.

Copyright © 2025, Oracle and/or its affiliates.

Xunit Processing

Xunit runs synchronously with the user OpChainProcessor thread in Xstore PUS; therefore, it does not generally suffer from timing issues where the system is not ready for the testing process or vice-versa. When Xstore PUS reaches a state where the user would be prompted for input, the next test step is prepared by Xunit. Just before the OpChainProcessor is ready to return control of the system to the user, an Xstore PUS event for the test step is readied and dispatched by the Xunit test engine.

Data Types

XunitConfig.xml is agnostic to data types. All data values, field values, and the like are configured as strings. When executing test cases, Xunit will automatically "cast" configured values to the data types expected by the prompts and forms with which the test engine interacts.

Testers and implementers do not need to concern themselves with data types when configuring Xunit test cases.

Copyright © 2025, Oracle and/or its affiliates.

2
Enabling and Configuring Xunit

The Xunit test automation engine is installed alongside Xstore POS. This chapter describes how to enable Xunit for use and perform base configuration on it.

Enable Xunit

To enable Xunit, add :test to the xstore.config.path.global.extensions= property in the configPath.properties file.

Enabling Recording

To enable Xunit to record automated tests, add :test:test/record to the xstore.config.path.global.extensions= property in the configPath.properties file.

Xunit Configuration

This section describes the properties used to configure Xunit processing.

system.properties

The following table describes the system.properties settings that define Xunit processing.

Tip: Many of the settings below can be controlled at runtime through the Settings tab in the Xunit Console.

Table 2-1 system.properties Settings

Property	Use
xunit.tags.disabled	This comma-delimited list of values determines which tag or suite names will be disabled at runtime. All test suites tagged with any value in this list, or having a name matching any value in this list will not run. Note that any value in both this list and the enabled list (see xunit.tags.enabled) will be disabled. xunit.tags.disabled=GIFT_CARD See Including and Excluding Tags - Configuration for recommended practices when enabling and disabling tags.

Copyright © 2025, Oracle and/or its affiliates.

Table 2-1 (Cont.) system.properties Settings

Property	Use
xunit.tags.enabled	This comma-delimited list of values determines which tag or suite names will be disabled at runtime. All test suites tagged with any value in this list, or having a name matching any value in this list will run. Note that any value in both this list and the disabled list (see xunit.tags.disabled) will be disabled. xunit.tags.enabled=LAYAWAY, TENDERING See Including and Excluding Tags - Configuration for recommended practices when enabling and disabling tags.
xunit.autorun	If this setting is set to true and Xunit is in test mode, Xunit will start automatically when Xstore POS starts. Valid values: true - Xunit will start automatically. false - Xunit will need to be started manually. xunit.autorun=true
xunit.captureonfail	If this setting is set to true, Xunit will save a screen capture to the export directory (see xunit.exportdir) when a test case fails. Valid values: true - A screen capture will be saved. false - No screen capture will be saved. xunit.captureonfail=true
xunit.debug	Determines whether source logging will be performed for every test step. Source logging is useful for tracing the exact path of a test case. Valid values: true - Source logging will be performed for every test step. false - Source logging will not be performed for every test step. xunit.debug=true
xunit.delay	Sets the step delay (in milliseconds) when executing in TEST mode. Note that, in RECORD mode, Xunit will always play test cases back with a delay of 1000 milliseconds (one second). Maximum delay is 9999 milliseconds. xunit.delay=2000
xunit.exportdir	Determines the export directory to which recorded test scripts and screen captures are written. Default value is the Xstore POS runtime directory. xunit.exportdir=C:\\xunit-export
xunit.failuremode	Determines whether Xunit will halt immediately upon failing a test case. Valid values: HALT - Xunit will halt immediately upon failing a test. xunit.failuremode=HALT

Copyright © 2025, Oracle and/or its affiliates.

Table 2-1 (Cont.) system.properties Settings

Property	Use
xunit.noiterations	Determines whether Xunit will ignore any configured iteration values on RunTest elements when executing in TEST mode. Each test case will be executed only one time if this is set to true. Valid values: true - Each test case will only be executed once. false - If a test case is configured to be executed multiple times, it will run the configured number of times. xunit.noiterations=true
xunit.noreceipts	Determines whether receipts will print while test cases are running. Suppressing receipt printing speeds up Xunit testing and saves paper. Valid values: true - No receipts will be printed. false - Receipts will be printed as they would for normal operation. xunit.noreceipts=true
xunit.height	Determines the height of the Xunit window in TEST mode. xunit.height=275
xunit.width	Determines the width of the Xunit window in TEST mode. xunit.width=400
xunit.log.console.maxlines	Determines how many log lines will be displayed on the run tab. xunit.log.console.maxlines=2000

Including and Excluding Tags — Configuration

When enabling or disabling tags in the system.properties files, it is recommended that you use the following method to enable and disable tags:

1. Enable the main tags for the functional areas being tested.
2. Disable any sub-areas that should not be tested.

```
xunit.tags.enabled=REG_TRANS,LAYAWAY,SYSTEM_CYCLE,LOCAL_CURRENCY,CREDIT_CARD,CHECK
xunit.tags.disabled=LOYALTY,SUSPEND_RESUME,TENDER_EXCHANGE
```

log4j2-custom.xml

The following configurations should be added to the log4j2-custom.xml file. These configurations are used to generate the xunit.out file in the Xstore POS log directory.

Copyright © 2025, Oracle and/or its affiliates.

> **Note**
>
> The xunit.out file (as configured here) stores the console output produced by Xunit.

```
<appender name="XUNIT_OUTPUT_FILE"
class="dtv.util.log4j.DtvDailyRollingFileAppender">
  <param name="File" value="${user.dir}/log/xunit.out" />
  <param name="MaxBackupIndex" value="14" />
  <layout class="dtv.util.log4j.DtvPatternLayout">
    <param name="ConversionPattern" value="%d %m%n" />
  </layout>
</appender>
<Logger name="dtv.xunit.XunitOutputFile" additivity="false" level="info">
  <AppenderRef ref="XUNIT.file" />
</Logger>
<XstRollingRandomAccessFile name="XUNIT.file" fileName="${sys:user.dir}/${sys:log.dir.name}/xunit.out"
filePattern="${sys:user.dir}/${sys:log.dir.name}/xunit.%d{yyyy-MM-dd}.out">
   <PatternLayout pattern="%d  %m%n" />
   <TimeBasedTriggeringPolicy />
   <DefaultRolloverStrategy max="14" />
</XstRollingRandomAccessFile>
```

Action Logging

By setting the log level to `INFO` for the `dtv.pos.framework.action.ActionLogger`, the system's available actions and form fields are written to the Eclipse console whenever a user prompt is displayed. This logging helps facilitate test case writing.

To change the `ActionLogger` log level to `INFO`, add the following configuration to the log4j2-custom.xml file:

```
  <!-- INFO: Capture Xstore technical details for reference within Xunit test definition files -->
   <Logger name="dtv.pos.framework.action.ActionLogger" level="info" />
     <priority value="WARN" />
   </category>
```

```
WARN
******FORM ACTIONS******
FORM:
 CANCEL (Back)
 CONTINUE (Skip)
 ACCEPT (Process)
 HELP (Help)
FORM FIELD KEYS:
 employeeId
 customerId
 firstName
 lastName
 city
 state
```


Copyright © 2025, Oracle and/or its affiliates.

```
 country
 postalCode
 telephone
 loyaltyNumber
 commAcctNumber
 companyName
 storeNumber
 emailAddress :: dtv.ops.log.ActionLogger.logFormActions(ActionLogger.java:52) 2010-10-29 16:47:26,227
[REGISTER:OCP [User]]
WARN
******PROMPT ACTIONS******
PROMPT:
 CANCEL (Back)
 CONTINUE (Skip)
 ACCEPT (Process)
 HELP (Help)
MENU:
 CLOCK_IN_OUT (Clock In/Out)
 PRE_VIEW_TIMECARD (View Timecard)
 PRE_SCHEDULE_VIEW (View Schedule)
 PRE_SCHEDULED_EMPLOYEES_VIEW (View Scheduled Employees)
 CHANGE_EMPLOYEE_PASSWORD (Change Password)
 GIFT_RECEIPT_REISSUE_SELECTION (Reissue Gift Receipt)
 REPRINT_LAST_RECEIPT (Reprint Last Receipt)
 VOUCHER_BALANCE_INQUIRY (Gift Card Balance Inquiry)
 VOUCHER_BALANCE_INQUIRY (Merch Credit Card Balance Inquiry)
 VOUCHER_BALANCE_INQUIRY (Paper Store Credit Balance)
 CREDIT_ACCT_LOOKUP (Private Credit Account Lookup)
 GIFT_RECEIPT_PRICE_LOOKUP (Gift Price Lookup)
 TRAINING_MODE_ENTER_LOGIN_MENU (Enter Training Mode)
 EMPLOYEE_TASKS_LOGIN (Employee Tasks)
 GENERIC_ITEM_LOOKUP (Item Lookup)
 REGISTER_OPTIONS_LOGIN (Manage Tills)
 FLASH_SALES_REPORT_VER2 (Flash Sales)
 PRE_CHANGE_APPLICATION (Back Office) ::
dtv.ops.log.ActionLogger.logAllActions(ActionLogger.java:46) 2010-10-29 16:47:26,227 [REGISTER:OCP [User]]
```

Mock Responders for Xstore POS

Xunit can be configured to simulate responses from remote services such as Order Broker Cloud Service or Oracle Retail Customer Engagement Cloud Service. This allows Xunit to test use of data from these services without requiring an actual connection.

To test with the mock service responses available in base Xunit:

1. Ensure that the configuration path includes both `:test` and `:test/{service}` (where `{service}` is the name of the remote service that is being simulated.
2. Ensure that the remote service is enabled in Xstore POS.
3. Run the associated Xunit tag containing the tests to be executed.

Copyright © 2025, Oracle and/or its affiliates.

3
Creating and Configuring Test Cases

This chapter describes the creation and configuration of test cases that are used by Xunit.

Recording Test Cases

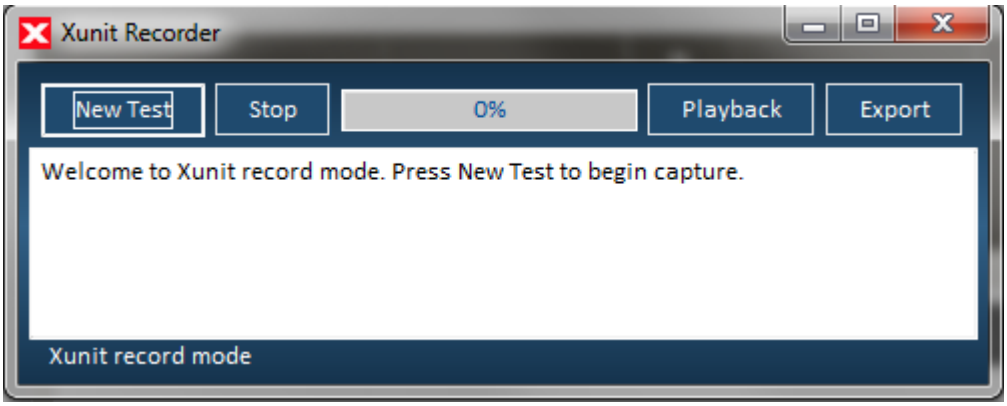
Xunit can record user actions and create test cases from those recorded actions. To use this feature, do the following:

1. If necessary, run Xunit in RECORD mode:

1. If necessary, run Xunit in RECORD mode:
   a. Set Xunit to RECORD mode (in the configPath.properties file).
   b. If necessary, stop Xstore POS.
   c. Start Xstore POS.

      The Record Console opens.

Figure 3-1 Xunit Recorder Console



2. Click the **New Test** button.

> **Note**
>
> Any action performed in the Xstore POS interface is recorded by Xunit as part of the test. Card swipes, however, are not included.

3. Perform the actions required by the test.
4. Click **Stop** when the test case is complete.

Copyright © 2025, Oracle and/or its affiliates.

> **Note**
>
> It is recommended that you continue the test until the transaction is complete and Xstore POS returns to a user login prompt.

5. If necessary, repeat steps 2-4 for each test case to include in the test suite.

> **Note**
>
> To run all the test cases recorded so far, click the **Playback** button. Xunit will load the captured events and run them as a test cycle.

6. Click the **Export** button when all the test cases have been captured.

   Xunit saves the stored test cases as an XML file in the export directory (see system.properties).

7. Repeat steps 2-6 for each XML test file to create.
8. Edit the test cases as necessary. See Xunit Configuration.

Example RECORD Mode Output File

Below is an example XML output file generated by the Xunit recorder.

```
<?xml version= 1.0 encoding= ISO-8859-1 ?>
<XunitConfig>
<Parameter name= XunitDelay value= 1000 />
<TestSuite name= MY_TEST_SUITE >
<RunTest description= Run test case TEST_CASE.1 testName= TEST_CASE.1 />
</TestSuite>
<TestCase applicationKey= REGISTER name= TEST_CASE.1 >
<TestStep action= ACCEPT data= 1 description= Enter 1 (PromptUserIdOp) />
<TestStep action= ACCEPT data= y description= Enter y (PromptPasswordOp) />
<TestStep action= ACCEPT description= Ok (PromptCommAssocListOp) >
<ListSelect index= 2 />
</TestStep>
<TestStep action= ACCEPT description= Process (CustomerSearchOp) >
<Form name= CUSTOMER_SEARCH >
<Field name= lastName value= Smith />
</Form>
</TestStep>
<TestStep action= ACCEPT description= Ok (CustomerSearchOp) >
<ListSelect index= 1 />
</TestStep>
<TestStep action= ACCEPT data= 1000141 description= Enter 1000141
(PromptItemScanOp) />
<TestStep action= ACCEPT description= Ok (PromptItemScanOp) />
<TestStep action= TENDER_LOCAL_CURRENCY description= Cash (DoNothingOp) />
<TestStep action= ACCEPT data= 5.00 description= Enter 5.00
(PromptTenderAmtOp) />
</TestCase>
</XunitConfig>
```


Copyright © 2025, Oracle and/or its affiliates.

Action Logging

Action logging can be used to determine the values that can be used for test cases. Action logging creates a record of active prompts, forms, form fields and menus in the xstore.log file.

> **Note**
>
> Action logging should only be enabled for as long as it is needed. Leaving this logging in place for prolonged periods will result in very large xstore.log files that will be difficult to archive and review.

Enable Action Logging

To enable action logging:

1. Stop Xstore POS.
2. Open the log4j2-custom.xml file.

   The default location for this file:

   - Windows: C:\xstoredata\xstore\cust_config\log4j2-custom.xml
   - Linux: /opt/xstoredata/xstore/cust_config/log4j2-custom.xml
1. Make the changes described in Action Loggingin Chapter 2, Enabling and Configuring Xunit.
2. Save the file.
3. Store Xstore POS.

Disable Action Logging

To disable action logging:

1. Stop Xstore POS.
2. Open the log4j2-custom.xml file.

   The default location for this file:

   - Windows: C:\xstoredata\xstore\cust_config\log4j2-custom.xml
   - Linux: /opt/xstoredata/xstore/cust_config/log4j2-custom.xml
3. Remove the changes described in Action Loggingin Chapter 2, Enabling and Configuring Xunit.
4. Save the file.
5. Store Xstore POS.

Xunit Configuration

Copyright © 2025, Oracle and/or its affiliates.

This section describes the elements and attributes used in Xunit configuration files.

Parameter

The <Parameter> element configures some basic configurations of the test.

- name - Name of the parameter.
- value - Value of the parameter.

The value of a <Parameter> can be retrieved by a reference to the name of the <Parameter> (see Referencing a Parameter).

Example <Parameter>

```
<!-- Number of milliseconds to wait before executing each step -->
<Parameter name="XunitDelay" value="1000" />
<!-- Other global params -->
<Parameter name="userid" value="1" />
<Parameter name="password" value="y" />
```

Referencing a Parameter

The value of a <Parameter> element can be retrieved by a reference to the name of the parameter:

```
!{parameter_name}
```

where parameter_name is the name of the parameter.

Example <Parameter> References

```
<TestStep description= Enter user ID action= ACCEPT data= !{userid} /> <TestStep description= Enter password action= ACCEPT data= !{password} />
```

DataPool

<DataPool> elements define collections of data that can be used for testing (for example, files or databases). <DataPool> elements support the following attributes:

- name - The name used to access the data pool with the test cases.
- queryKey - For query-based data pools, the query key (configured in QueryConfig.xml) used to populate the data pool. Note that the result class used for the query must implement the IOpsGenericQueryResult interface.
- filePath - For file-based data pools, the path to the flat file used to populate the data pool. See File Data Pool for more information about file-based data pools.
- maxRows - The maximum number of rows to retrieve from the data source. Default value is 50 rows.
- class - The implementation class to use for the data pool. The data pool class must implement IDataPool. It is preferred that the class extend AbstractDataPool.
- enabled - Boolean flag that determines whether the data pool is enabled. Default value is true.

Copyright © 2025, Oracle and/or its affiliates.

> **Note**
>
> This setting can be used to disable a base data pool in a customer overlay.

- forceLoad - Boolean flag that determines whether the data pool should be loaded automatically when Xunit launches. If this attribute is set to false, the pool will be loaded on first use. Default value is false.

Example <DataPool> Elements

```
<DataPool name= ITEMS queryKey= XUNIT_ITEMS maxRows= 200 >
<Parameter name= promptForQuantityFlag value= 0 />
<Parameter name= promptForPriceFlag value= 0 />
<Parameter name= promptForWeightFlag value= 0 />
<Parameter name= promptForDescriptionFlag value= 0 />
<Parameter name= warrantyFlag value= 0 />
<Parameter name= attachedItemsFlag value= 0 />
<Parameter name= serializedItemFlag value= 0 />
</DataPool>
<DataPool name= ENTER_USER filePath= test\xunit\datapool\user_list.txt />
```

DataPool Queries

Database queries used by <DataPool> elements should use the following ORDER BY clause:

```
ORDER BY COALESCE(t.update_date, t.create_date) DESC
```

By using this ORDER BY clause, testers only need to refresh the corresponding data pool and examine the first row returned.

Example Refreshing and Retrieving from a Data Pool

The following XML configurations shows how a DataPool is refreshed and the first row retrieved from the results.

```
<TestCase name= CUSTOMER_CREATE applicationKey= BACK_OFFICE >
<Parameter name= lname value= %?ENTER_CUSTOMER.LastName />
<Parameter name= fname value= %?ENTER_CUSTOMER.FirstName />
<!-- Test sequences to create a new customer in back office -->
...
<!-- Refresh the CUSTOMERS data pool and validate the most recent entry -->
<Assertion target= %^CUSTOMERS.LastName! compareType= EQUALS
compareVal= !{lname} />
<Assertion target= %^CUSTOMERS.FirstName compareType= EQUALS
compareVal= !{fname} />
</TestCase>
```


Copyright © 2025, Oracle and/or its affiliates.

> **Note**
>
> In the configuration line<Assertion target="%^CUSTOMERS.LastName!"... the exclamation point (!) at the end of the target attribute refreshes the DataPool. Note that the following line does not include an exclamation point. See % Data Pool Accessor for more information.

File Data Pool

File-based data pools must contain a comma-separated list of field names in the first line and comma-separated data rows in all subsequent lines.

Example File Data Pool Contents

```
userid,password,fname,lname
1,y,System,User
2,1,John,Smith
100,a1111111,Bob,Ross
```

TestStep

<TestStep> elements define single execution steps within test cases. Each <TestStep> represents a single, specific action or event to be dispatched to Xstore POS by the test case.

<TestStep>

elements support the following attributes:

- description - Description of the test step. This description will be written to the Xunit console when the test step is executed.
- action - The action to send to Xstore POS.
- data - The data dispatched to Xstore POS by the action.
- sleep - The number of milliseconds Xunit will sleep before executing the test step.
- timeLimit - The number of milliseconds that Xunit will wait for the test step to be completed. If Xstore POS takes longer than that to respond, Xunit will fail the test.
- condition - The conditional expression that must evaluate to **true** for the step to execute. If the condition evaluates to **false**, the <TestStep> will be skipped. See <TestStep> With Condition Attribute.
- stub - The fully qualified class name for the stubbed operation that must be running for the test step to execute.

> **Note**
>
> This attribute is typically only used when defining a test step for a non-applicable or non-user interactive operation that only displays a prompt when running as an "XFlow" design stub.

Copyright © 2025, Oracle and/or its affiliates.

The <TestStep> element includes the following subelements:

- <Form> - Used to enter data into form fields. See Form.
- <ListSelect> - Used to make a selection from a list. See ListSelect.

Example <TestStep>

```
<TestStep description="Enter item ID" action="ACCEPT" data="1002" />
```

Example <TestStep> With Condition Attribute

```
<!-- The below step should only execute when there are multiple line items on the
transaction;
if there is only one, the user will not be prompted to make a selection. -->
<TestStep description="Select an item" action="ACCEPT"
condition="@promptEquals(CHANGE_ITEM)">
...
</TestStep>
```

Using Conditions

Conditional test steps should be used sparingly, and are generally more appropriate in base test cases than in a customer overlay. This is because base test cases need to accommodate a wide variety of configuration options. Without conditional steps, many base test sequences would need to be overridden in an overlay.

> **Note**
>
> As a rule, test cases defined in the customer overlay should NOT include conditional steps.

Writing Strong Conditions

In the rare case that a conditional test step is necessary in an overlay, conditions should be both strong and reusable.

Example Weak Condition

The following example is a weak condition. It allows the test step to be skipped if the system has not achieved a certain state, without any regard for whether the system should have achieved that state.

```
<TestSequence name="LAYAWAY_PICKUP">
<TestStep description="Select Pickup Item" action="LAYAWAY_PICKUP_ITEM" />
<!-- Only run this step if partial pickups are allowed -->
<TestStep description="Select Pickup All" action="SELECT_ALL"
condition="@promptEquals(LAYAWAY_ITEMS_PICKUP_LIST)" />
<TestStep description="Select Add Tenders" action="LAYAWAY_PRE_TENDERING" />
</TestSequence>
```

Example Strong Condition

Copyright © 2025, Oracle and/or its affiliates.

The following example is a strong condition because it checks SystemConfig.xml to determine whether the layaway partial pickup step should be executed.

```
<TestSequence name="LAYAWAY_PICKUP">
<TestStep description="Select Pickup Item" action="LAYAWAY_PICKUP_ITEM" />
<!-- Only run this step if partial pickups are allowed -->
<TestStep description="Select Pickup All" action="SELECT_ALL"
condition="@invokeConfigMgr(allowPartialLayawayPickups)" />
<TestStep description="Select Add Tenders" action="LAYAWAY_PRE_TENDERING" />
</TestSequence>
```

Form

<Form> elements are used to enter data into form fields during a test step. The <Form> element includes the following attributes:

- name - The form key on which to invoke the test step action and set field data.

The <Form> element contains one or more of the following subelement:

- Field - Defines how the data is entered into the field.

Field

The <Field> element defines the field into which data is entered and the data that is entered into the field. The <Field> element uses the following attributes:

- name - ID of the field into which the data is entered.
- value - Defines the data entered into the field.

Example <Form> Element With <Field> Subelements

```
<TestStep description="Search for a customer" action="ACCEPT">
  <Form name="CUSTOMER_SEARCH">
  <Field name="lastName" value="Doe" />
  <Field name="firstName" value="John" />
  </Form>
</TestStep>
```

ListSelect

<ListSelect> elements select one or more rows from an Xstore POS list prompt. <ListSelect> elements support the following attributes:

- name - The method to invoke on each element in the list to check for a valid selection
- value - The method comparison value for a single-select list.
- values - A comma-delimited list of method comparison values for a multi-select list.
- modelKey - The Xstore POS list model to use for the selection. Default list prompt model will be used if this attribute is not set.
- index - The one-based row index that will be selected.

Copyright © 2025, Oracle and/or its affiliates.

Example <ListSelect> Select By Method

```
<ListSelect> Select By Method
<TestStep description="Select the commissioned associate with employee ID = 1" action="ACCEPT">
  <ListSelect method="getEmployeeId" value="1" />
</TestStep>
```

Example <ListSelect> Select By Index

```
<TestStep description="Select the second item in the list" action="ACCEPT">
  <ListSelect index="2" />
</TestStep>
```

Example <ListSelect> Multi-Select List

```
<TestStep description="Select items 1002, 1003, and 1004 from the gift receipt item list" action="ACCEPT">
  <ListSelect method="getItemId" values="1002, 1003, 1004" />
</TestStep>
```

Example <ListSelect> Transaction List Model

```
<!-- Change quantity -->
<TestStep description="Change item quantity" action="PRE_CHANGE_ITEM_QUANTITY" />
<TestStep description="Select line for quantity change" action="ACCEPT">
  <ListSelect method="getItemId" value="1003" modelKey="CURRENT_TRANSACTION" />
</TestStep>
<TestStep description="Enter new quantity" action="ACCEPT" data="2" />
```

Assertion

`<Assertion>` elements verify specific conditions at runtime. If the configured assertion does not evaluate to true, the test case fails.

> **Note**
>
> Assertions should occur frequently over the course of a test. Without periodic validations that data is handled correctly, test cases will not be able to pinpoint where errors occur.

Basic Assertion

Basic assertions are Boolean comparisons that are part of the core Xunit framework. They are configured through the following attributes:

- `target` - The basis for the comparison.
- `compareType`- The type of comparison to perform. This attribute has the following possible values:
  - IS_NULL

Copyright © 2025, Oracle and/or its affiliates.

  - IS_EMPTY
  - EQUALS
  - GREATER_THAN
  - LESS_THAN
- compareVal- The value to be compared against the target to validate the assertion.

> **Note**
>
> This attribute should be omitted when using the IS_NULL or IS_EMPTY comparison type.

- invert - Determines whether the result of the comparison is inverted. In other words, this adds a "NOT" to the compareType. This attribute allows for comparisons such as "NOT_LESS_THAN" or "IS_NOT_NULL"

  Default value is false.

Example Basic <Assertion>

This example will pass only if the current transaction total is less than 1000.

```
<Assertion target="@getTransaction.getTotal" compareType="LESS_THAN" compareVal="1000" />
```

Prompt Assertion

Prompt assertions validate that a prompt is displayed to the user. This element supports the following attribute:

- promptKey- ID of the prompt that should be displayed to the user.

Example Prompt <Assertion>

```
<Assertion promptKey="LOGIN_USER_ID" />
```

Custom Assertion

Xunit provides the ability to define custom assertion classes implemented by the tester. Customer assertion classes must extend the AbstractAssertion class. Customer assertions are configured through <Parameter> elements (see Parameter).

Customer assertions support the following attributes:

- class - The class implemented by the assertion.
- invert - Determines whether the result of the comparison is inverted. In other words, this adds a "NOT" to the result returned by the class.

Example Custom <Assertion>

```
<Assertion class="dtv.xunit.assertion.StockLevelAssertion" invert="true">
  <Parameter name="ITEM_ID" value="1002" />
  <Parameter name="BUCKET_ID" value="ON_HAND" />
  <Parameter name="LOCATION_ID" value="DEFAULT" />
```


Copyright © 2025, Oracle and/or its affiliates.

```
  <Parameter name="UNIT_COUNT" value="99" />
</Assertion>
```

Writing Strong Assertions

An <Assertion> should be written so that it validates the application state that the sequence is working to achieve. This allows an <Assertion> to be reused anywhere without concern for the surrounding application flow.

Example Weak Assertion

```
<TestSequence name="LOGIN" parameters="userid, password">
  <!-- Enter user ID and password -->
  <TestStep description="Enter user ID !{userid}" action="ACCEPT" data="!{userid}" />
  <TestStep description="Enter password !{password}" action="ACCEPT" data="!{password}" />
  <!-- Make sure the system is on the item scan prompt! -->
  <Assertion promptKey="SALE_ITEM_SCAN" />
</TestSequence>
```

The preceding example is a weak <Assertion> because it does not actually check whether the employee is successfully logged in. It only checks whether the item scan prompt is currently displayed. If Xstore POS logged in the wrong employee (or no employee at all), the <Assertion> would not know that it needs to fail the test case.

Furthermore, the <Assertion> is not sufficiently reusable for the following reasons:

- The <Assertion> makes assumptions about the application flow that may not be valid. For example, if the test sequence logs user into the back office or till options, the test case would fail because the item scan prompt would not be displayed after logging in.
- If a new prompt is inserted prior to beginning a sale (for example, a customer search or the selection of a commissioned associate), the test sequence would need to be updated. Otherwise, either the test sequence would need to be updated or every test case using it would fail.

Example Strong Assertion

```
<TestSequence name="LOGIN" parameters="userid, password">
  <!-- Enter user ID and password -->
  <TestStep description="Enter user ID !{userid}" action="ACCEPT" data="!{userid}" />
  <TestStep description="Enter password !{password}" action="ACCEPT" data="!{password}" />
  <!-- Make sure the employee is logged in! -->
  <Assertion target="@getCurrentEmployeeId" compareType="EQUALS" compareVal="!{userid}" />
</TestSequence>
```

The preceding example is a strong <Assertion> because it validates the application state that the sequence is intended to achieve. This assertion also makes the test sequence reusable because it can be reused anywhere without concern for the surrounding application flow.

Verifying Log File Creation

To verify that data was appended to the PosLog.xml file upon completing a transaction, do the following:

Copyright © 2025, Oracle and/or its affiliates.

Example POSLog Validation

```
<TestCase name="POSLOG_TEST">
  <Parameter name="oldFileSize" value="@getLogFileSize(POSLOG)" />
  <SequenceRef name="LOGIN" values="1, y" />
  <!-- Ring transaction -->
  ...
  <!-- Make sure POSLog size has increased-->
  <Assertion target="@getLogFileSize(POSLOG)" compareType="GREATER_THAN" compareVal="!
{oldFileSize}" />
</TestCase>
```

TestSequence

A test sequence is a sequential set of test steps that can be reused in any number of Xunit test cases. <TestSequence> elements support the following attributes:

- name - Name of the <TestSequence>. This is used by <SequenceRef> elements to refer to the <TestSequence> (see SequenceRef).
- parameters - A comma-delimited list of the names of <Parameter> elements used within the test sequence. See Parameter.

> **Note**
>
> Any <Parameter> used within the <TestSequence> must be listed in this attribute. If a <Parameter> name is reference within the <TestSequence>, but is not listed in this attribute setting, the test sequence will fail.

A <TestSequence> can include the following subelements, which are performed sequentially:

- <TestStep> - Test steps performed within the <TestSequence>. See TestStep.
- <SequenceRef> - A reference to another <TestSequence>, which will be performed, in its entirety, as part of the current <TestSequence>. See Sequence Ref.

Example <TestSequence>

```
<!-- Log into the register or back office -->
<TestSequence name="LOGIN" parameters="userid, password">
  <TestStep description="Enter user ID !{userid}" action="ACCEPT" data="!{userid}" />
  <TestStep description="Enter password !{password}" action="ACCEPT" data="!{password}" />
</TestSequence>
```

SequenceRef

The <SequenceRef> element calls a <TestSequence> as part of a <TestCase> (see TestCase), or as part of a separate <TestSequence> (see TestSequence). The <SequenceRef> element supports the following attributes:

- name - The name of the <TestSequence> being called.

Copyright © 2025, Oracle and/or its affiliates.

- values - A comma-delimited list of arguments that will be used by the parameters called by the <TestSequence>. Each value will be called sequentially by the parameters as they are called within the <TestSequence>.
- iterations - The number of times to run the <TestSequence>.

Example <SequenceRef>

```
<SequenceRef name="SELL_ITEM" values="1002" iterations="2" />
```

TestCase

> **Note**
>
> Test cases should continue through a transaction until the transaction completes. When a test case completes before the end of a transaction, Xunit cannot spot errors that only appear later in the transaction process.

The <TestCase> element defines a sequential set of steps and assertions that represent a single Xunit test case that can pass or fail. The <TestCase> element supports the following attributes:

- name - The name of the <TestCase>. This is used by the <TestSuite> element to create an entire test suite.
- applicationKey = The application mode in which the test case should run. Default value is REGISTER. This attribute has the following possible values:
  - REGISTER
  - BACK_OFFICE
- assertTypes- A comma-delimited list of <AssertType> elements to check after the test case completes. See AssertType for more information. See Verifying Transaction Completion for more information about how they are used.

This element supports the following subelements:

- <TestStep> - Defines a test step performed by the <TestCase>. See TestStep.
- <SequenceRef> - Refers to a <TestSeq> called by the <TestCase>. See SequenceRef and TestSequence.
- <Assertion> - An <Assertion> checked by the <TestCase>. See Assertion.
- <Parameter> - A basic configuration of the test. See Parameter.

> **Note**
>
> It is recommended that test cases only include references to test sequences. This makes it easier to configure overrides to test cases.

Example <TestCase>

```
<TestCase name="SALE_TRANS" applicationKey="REGISTER">
 <SequenceRef name="LOGIN" values="1,y" />
```


Copyright © 2025, Oracle and/or its affiliates.

```
  <SequenceRef name="BEGIN_TRANS_NO_CUST" values="1" />
  <SequenceRef name="SELL_ITEM" values="1002" />
  <TestStep description="Change item price" action="PRE_CHANGE_ITEM_PRICE" />
  <TestStep description="Enter new price" action="ACCEPT" data="9.99" />
  <SequenceRef name="SELECT_REASON_CODE" values="03" /> <!-- Wrong retail in POS -->
  <!-- Make sure the transaction total is less than $1000 -->
  <Assertion target="@getTransaction.getTotal" compareType="LESS_THAN" compareVal="1000" />
  <TestStep description="Begin tendering" action="ACCEPT" />
  <TestStep description="Tender to credit card" action="TENDER_CREDIT_CARD" />
  <TestStep description="Enter card number" action="ACCEPT" data="4444333322221111" />
  <TestStep description="Enter expiration date" action="ACCEPT" data="1012" />
  <SequenceRef name="ENTER_TRANS_TOTAL" />
  <TestStep description="Enter CID number" action="ACCEPT" data="123" />
  <TestStep description="Is sale complete prompt" action="YES" />
</TestCase>
<TestCase name="LAYAWAY_SETUP" assertTypes="LAYAWAY, COMPLETED_TRANS">
  ...
</TestCase>
```

Verifying Transaction Completion

The Xunit assertTypes attribute provides a shorthand way of verifying the result of a

```
<TestCase>
```

element:

Example Validate the End of a Transaction

```
<!-- Make sure a new transaction is persisted to the DB at the end of this test case -->
<TestCase name="TRANS_TEST" assertTypes="COMPLETED_TRANS">
  <SequenceRef name="LOGIN" values="1, y" />
  <!-- Ring transaction -->
  ...
</TestCase>
```

Example Validate Layaway Creation and the End of a Transaction

```
<!-- Make sure a new transaction and layaway are persisted to the DB at the end of this test case -->
  <TestCase name="LAYAWAY_TEST" assertTypes="LAYAWAY, COMPLETED_TRANS">
  <SequenceRef name="LOGIN" values="1, y" />
  <!-- Setup new layaway account -->
  ...
</TestCase>
```

Deprecated Method

Previously, transactions were verified by configuring an `<Assertion>` to directly check whether data from the completed transaction could be found in the database. The new `assertTypes` attribute replaces this method.

TestSuite

Copyright © 2025, Oracle and/or its affiliates.

The <TestSuite> element defines sets of test cases that run as ordered groups. The <TestSuite> element supports the following attributes:

- name - The name of the test suite. The name also functions as a tag.
- tags - A comma-delimited list of keywords associated with the <TestSuite>. These values are used by unit.tags.enabled and xunit.tags.disabled settings in the system.properties file.

The <TestSuite> element supports the following subelement:

- <RunTest> - Refers to a <TestCase> run by the <TestSuite>. See RunTest.

Example <TestSuite>

```
<TestSuite name="SALE_RETURN" tags="CRITICAL, REGRESSION, SALE, RETURN">
  <RunTest testName="SALE_TRANS" description="Complete a basic sale transaction" iterations="3" />
  <RunTest testName="VERIFIED_RETURN_TRANS" description="Complete a verified return transaction"
reset="false" />
</TestSuite>
```

Run Test

The <RunTest> element is used by the <TestSuite> element to call a <TestCase> that is run as part of the suite of tests. The <RunTest> element supports the following attributes:

- testName- The name of the <TestCase> (see TestCase) being called by a <TestSuite> (see TestSuite).
- description - Description of the <TestCase>. This description is written to the Xunit console when the test case is executed.
- reset - If set to false, the test engine will not return the Xstore POS application to the login screen upon successful completion of the test case. Default value is true.

> **Note**
>
> A system reset will always occur when the test case fails, even when this attribute is set to false.

- iterations - The number of times the test case will run.

Example RunTest>

```
  <RunTest testName="SALE_TRANS" description="Complete a basic sale transaction" iterations="3" />
  <RunTest testName="VERIFIED_RETURN_TRANS" description="Complete a verified return transaction"
reset="false" />
```

Excetpion

The <Exception> element is used to react to conditional prompts that may appear during the course of executing a test case. Typically these are used to bypass data-dependent system notifications that otherwise have no impact on the outcome of the test.

For example, capturing the EXCEED_MAX_CASH_AMOUNT prompt ensures that the test case won't fail due to an unexpected prompt when the till has exceeded its configured cash

Copyright © 2025, Oracle and/or its affiliates.

threshold. If Xunit encounters this prompt at any point (or multiple points) while processing test cases, the test engine will respond with the configured action, then move on.

Another common use for an <Exception> element is to acknowledge INSUFFICENT_QUANTITY messages that are dependent on stock levels in the Xstore POS database.

> **Note**
>
> See Appendic C Global Exceptions for a full list of exceptions available in Xunit.

The <Exception> element supports the following attributes:

- name - Name of the Xstore POS prompt or form key that will trigger the <Exception>.
- action - The action used to respond to the prompt.
- data - The data sent to Xstore POS when the prompt appears.
- enabled - Boolean flag that determines whether the <Exception> is enabled. Default value is true.

Example <Exception>

```
<!-- Global exceptions -->
<Exception name="EXCEED_MAX_CASH_AMOUNT" action="ACCEPT" />
```

Disabling an Exception

Sometimes a tester may wish to actually test and assert a specific prompt normally bypassed using an exception. Xunit provides the ability to disable a globally-defined <Exception> element at the test case level. This is done by creating an exception with its enabled attribute set to false.

Example Disabling an <Exception>

```
<!-- By default, this exception is in effect for all test cases -->
<Exception name="EXCEED_MAX_CASH_AMOUNT" action="ACCEPT" />

<TestCase name="MAX_CASH_AMOUNT_TEST">

  <!-- Test steps go here  -->
  ...

  <!-- The EXCEED_MAX_CASH_AMOUNT exception will NOT be in effect for the enclosing test case  -->
  <!-- This is accomplished by setting the attribute enabled="false" -->
  <Exception name="EXCEED_MAX_CASH_AMOUNT" action="ACCEPT" enabled="false" />
</TestCase>
```

AssertType

The <AssertType> element defines sequence-based validation logic to be run at the end of specific test cases. This simplifies the syntax needed to validate that data has been successfully saved to the database.

Copyright © 2025, Oracle and/or its affiliates.

Each global <AssertType> element sets the parameters for a specific type of validation. <TestCase> elements can then include a comma-delimited list of <AssertType> names using the assertTypes attribute (see TestCase). The <AssertType> element supports the following attributes:

- name - The name used to refer to the <AssertType>.
- sequenceType - The Xstore POS sequence type from which Xunit will check the next value at the beginning of any test case using the <AssertType>.
- evaluator - The dynamic Xunit expression to evaluate at the end of any <TestCase> for which the <AssertType> is being used. Xunit will compare the result to the previously checked sequence value; if the values do not match, the test case will fail.

Example <AssertType>

```
<AssertType name="COMPLETED_TRANS" sequenceType="Transaction"
evaluator="%^COMPLETED_TRANS.TransSeq!" />
<AssertType name="LAYAWAY" sequenceType="LAYAWAY"
evaluator="%^UNPAID_LAYAWAYS.CustAcctId!" />
```

Base AssertTypes

There are some AssertTypes available in base Xunit.

Table 3-1 Base Assert Types

Name	Description of Data Checked	Table Checked
CANCELLED_TRANS	Cancelled transaction.	trn_trans
COMPLETED_TRANS	Completed transaction.	trn_trans
CUSTOMER	New customer record.	crm_party
LAYAWAY	New layaway account.	cat_cust_acct
ON_HOLD	On hold transaction.	trn_trans
ORDER	New order.	xom_order
PRESALE	Presale transaction.	trn_trans
RAINCHECK	New rain check.	trn_raincheck
SEND_SALE	New send sale.	cat_cust_acct
SHIFT	New employee shift.	sch_shift
SP_ORDER_CUST	New account for a special order to be shipped to a customer.	cat_cust_acct
SP_ORDER_STORE	New account for a special order to be shipped to a store.	cat_cust_acct
SUSPENDED_TRANS	A transaction was suspended.	trn_trans
TASK	New task.	hrs_employee_task
TIMEOFF	Time off added for an employee.	sch_emp_time_off
WARRANTY	New warranty.	itm_warranty
WORK_ORDER	New work order.	cwo_work_order

Special Notations

Copyright © 2025, Oracle and/or its affiliates.

The following notations are used to reference information within test configuration elements.

!{...} Parameter Reference

The !{...} notation is used to reference data configured within <Parameter> elements. For more information about this notation, see Referencing a Parameter.

Example !{...}

```
<!-- Sequence for adding an item to the sale...
Must complete in under 5 seconds to prevent failure! -->
<TestSequence name="SELL_ITEM" parameters="itemid">
  <TestStep description="Enter item ID !{itemid}" action="ACCEPT" data="!{itemid}" timeLimit="5000" />
</TestSequence>
<!-- Sample test case -->
<TestCase name="BASIC_SALE_TRANS" applicationKey="REGISTER">
  <!-- Parameters -->
  <Parameter name="myitemid" value="1001" />
  <!-- Test steps -->
  <SequenceRef name="LOGIN" values="!{userid}, !{password}" />
  <SequenceRef name="ASSIGN_CUSTOMER" values="12345" />
  <SequenceRef name="SELL_ITEM" values="!{myitemid}" />
  ...
```

@ Evaluation Method Chain

The @ sign is used to invoke a period-delimited method chain that retrieves system information.

The first method in the chain must be an instance of the configured EvaluationMethods class. EvaluationMethods follows the singleton design pattern commonly seen in the Xstore POS "helper" classes, and can be extended in the customer overlay to provide customer-specific overrides and functionality.

Arguments can be passed into evaluation methods using parentheses and, for multiple-argument method calls, commas. See Example Passing an Argument.

> **Note**
>
> See Appendix B Evaluation Methods for a full list of evaluation methods available in Xunit.

Example Retrieving a Transaction Total

```
<!-- Enter the current transaction total -->
<TestSequence name="ENTER_TRANS_TOTAL">
  <TestStep description="Enter transaction amount" action="ACCEPT" data="@getTransaction.getTotal.abs" />
</TestSequence>
```


Copyright © 2025, Oracle and/or its affiliates.

Example Perform a Verified Return Against the Previously-completed Sale

```
<TestStep description="Enter original rcpt info" action="ACCEPT">
  <Form name="ORIGINAL_RECEIPT_INFORMATION">
    <Field name="origTransSeq" value="@getLastTransactionId.getTransactionSequence" />
    <Field name="origTransBusinessDate" value="@getLastTransactionId.getBusinessDate" />
    <Field name="origLocationId" value="@getLastTransactionId.getRetailLocationId" />
    <Field name="origWorkStationId" value="@getLastTransactionId.getWorkstationId" />
  </Form>
</TestStep>
```

Example Passing an Argument

```
<TestStep description="Enter new price" action="Ok" data="@getTransaction.getTotal.multiply(2)" />
```

% Data Pool Accessor

Data pool result fields are accessed using the percent sign (%), followed by an optional operator, followed by the data pool name, a period (.), then the field to be accessed. The currently-supported data pool operators are:

- ? Operator for using a random element from a data pool. This is also the default behavior when an operator is not supplied.
- ^ Operator for using the most recently accessed element from a data pool.
- + Operator for using the next sequential element in a data pool.
- - Operator for using the previous sequential element in a data pool.

> **Note**
>
> See Appendix A Data Pools for a full list of data pools and fields.

Example Data Pool Referencing

```
<!-- Adds 2 random merch items to the sale -->
<TestSequence name="SELL_RANDOM_MERCH_ITEM" parameters="itemid">
  <TestStep description="Enter item ID" action="ACCEPT" data="%ITEMS.ItemId" />
  <TestStep description="Enter item ID" action="ACCEPT" data="%?ITEMS.ItemId" />
</TestSequence>
```

Example Refreshing the Data Pool

Data pools are refreshed (that is, the queries are re-run) every time the test engine is restarted. Suffixing the data pool access expression with an exclamation point (!) will refresh the data pool prior to the access. For example, the following expression will refresh the data pool and return the first result, thereby returning the most recently suspended transaction's total for use in the comparison.

```
...
<SequenceRef name="RESUME_TRANS"/>
<Assertion target="@getTransaction.getTotal" compareType="EQUALS"
```


Copyright © 2025, Oracle and/or its affiliates.

```
compareVal="%^SUSPENDED_TRANS.Total!"/>
...
```

Spring Configurations

Xunit uses Spring beans for some testing functions. Configuration of these Spring beans is described in this section.

xunit-beans.xml

Spring beans exclusive to Xunit are defined in the xunit-beans.xml file.

Field Handlers

Xunit sets data on form fields by calling the setValue method of IEditModel. However, for a small subset of Xstore POS form models, this does not work properly. For these cases, Xunit provides the ability to delegate to custom field handler classes based on the displayed form key. Field handlers must implement the IFormFieldHandler interface, which defines methods needed for both executing and recording test steps.

Field handler Spring beans are assigned to specific form keys in the spring.xml file.

spring.xml

```
<!-- Form/field selection beans -->
<bean id="fieldHandlerRegistry" class="dtv.pos.util.field.FieldHandlerRegistry" />
<bean id="fieldHandlerFactory" class="org.springframework.beans.factory.config.ServiceLocatorFactoryBean">
  <property name="serviceLocatorInterface" value="dtv.pos.util.field.FieldHandlerFactory" />
  <property name="serviceMappings">
    <props>
      <prop key="default">defaultFieldHandler</prop>
      <prop key="AUTH_MORE_INFO">moreAuthInfoFieldHandler</prop>
      <prop key="INVENTORY_COUNT_SHEET">inventoryCountFieldHandler</prop>
      <prop key="PAYROLL_SUMMARY">tableCellFieldHandler</prop>
      <prop key="SCHED_SUMMARY">tableCellFieldHandler</prop>
      <prop key="TILL_COUNT_SUMMARY">countSummaryFieldHandler</prop>
    </props>
  </property>
</bean>

<!-- Prototype field handler beans -->
<bean id="defaultFieldHandler" class="dtv.pos.util.field.DefaultFieldHandler" scope="prototype">
  <property name="methodRegistry">
    <map>
      <entry key="dtv.util.ICode" value="getCode" />
      <entry key="dtv.pos.employee.SecurityGroupWrapper" value="getGroupId" />
      <entry key="dtv.xst.dao.crm.impl.CustomerAffiliationModel" value="getCustomerGroupId" />
      <entry key="dtv.xst.dao.inv.impl.InventoryLocationBucketModel" value="getBucketId" />
      <entry key="dtv.xst.dao.inv.impl.InventoryLocationModel" value="getName" />
      <entry key="dtv.xst.dao.inv.impl.InventoryValidDestinationsModel" value="getDestinationId" />
```


Copyright © 2025, Oracle and/or its affiliates.

```
      <entry key="dtv.xst.dao.sec.impl.GroupModel" value="getGroupId" />
    </map>
  </property>
 </bean>
 <bean id="countSummaryFieldHandler" class="dtv.pos.util.field.CountSummaryFieldHandler" scope="prototype" />
 <bean id="inventoryCountFieldHandler" class="dtv.pos.util.field.InventoryCountSheetHandler"
scope="prototype" />
 <bean id="moreAuthInfoFieldHandler" class="dtv.pos.util.field.MoreAuthInfoFieldHandler" scope="prototype" />
 <bean id="tableCellFieldHandler" class="dtv.pos.util.field.TableCellFieldHandler" scope="prototype" />
```

Select Methods

When populating a form field whose possible values are complex objects (such as a combo box or embedded list), an error similar to the following may occur if the object type is not registered in the methodRegistry map within the defaultFieldHandler Spring bean definition (see spring.xml).

```
CAUSE: java.lang.String [STORE_TRANSFER] is not assignable to a dtv.pos.inventory.type.InvDocType
```

To test these fields using Xunit, an entry for the class must be added to the bean definition. The entry will be a class/method pair that indicates to Xunit that the indicated method will be used to set form fields whose values are of the indicated type.

Skip Types

Specific prompts can be configured as skip types to prevent the test engine from attempting to execute test steps prematurely. For example, an authorization processing prompt will appear while an authorization is in process, and will eventually be replaced without any user action. A skip type ensures that Xunit does not fire a test case while this prompt appears on the screen.

To ensure that Xunit never fires test events at prompts like these, the prompt is blocked in the skipTypeRegistry bean definition in spring.xml:

```
<bean id="skipTypeRegistry" class="dtv.pos.util.field.SkipTypeRegistry">
  <property name="registry">
    <list>
      <value>AUTH_PROCESSING</value>
      <value>LABEL_LAYOUT_CALCULATING</value>
      <value>REPORT_RUNNING</value>
    </list>
  </property>
</bean>
```


Copyright © 2025, Oracle and/or its affiliates.

4
Running Tests

This chapter describes how to perform automated testing on Xstore POS through Xunit.

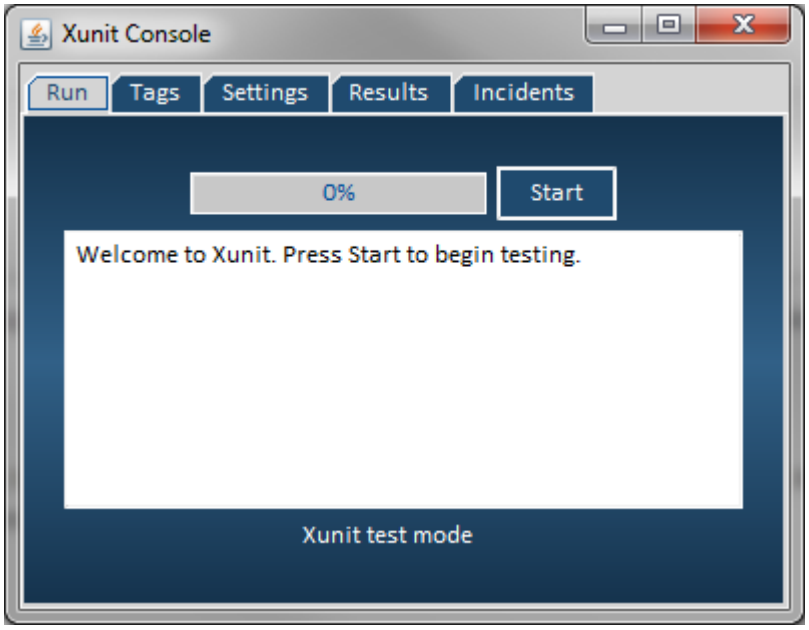
Xunit Console

Xunit is controlled through the Xunit Console. The Xunit Console can only be opened by configuring it to open automatically when Xstore POS opens (see xunit.autorun in Enabling and Configuring Xunit).

> **Note**
>
> If Xunit is not configured to run automatically at Xstore POS startup, it cannot be run.

The Xunit Console always remains on top of the Xstore POS screen.

Figure 4-1 Xunit Console



Tabs

The Xunit console includes the following tabs:

Copyright © 2025, Oracle and/or its affiliates.

- **Run** - Used to start and stop testing. Shows testing progress and a summary of the results. See Running Test Cases for information about this tab.
- **Tags** - Displays and edits the tags that are configured to run at startup.
- **Settings** - Controls behavioral settings for Xunit. The default value for each setting is preconfigured for each Xstore POS installation. The default settings are restored upon restarting the Xstore POS application.
- **Results** - Displays a summary of test results during and after testing.
- **Incidents** - Displays a list of incidents reported during execution, as well as a summary of the incidents upon test completion. See Incidents for more information about incidents.

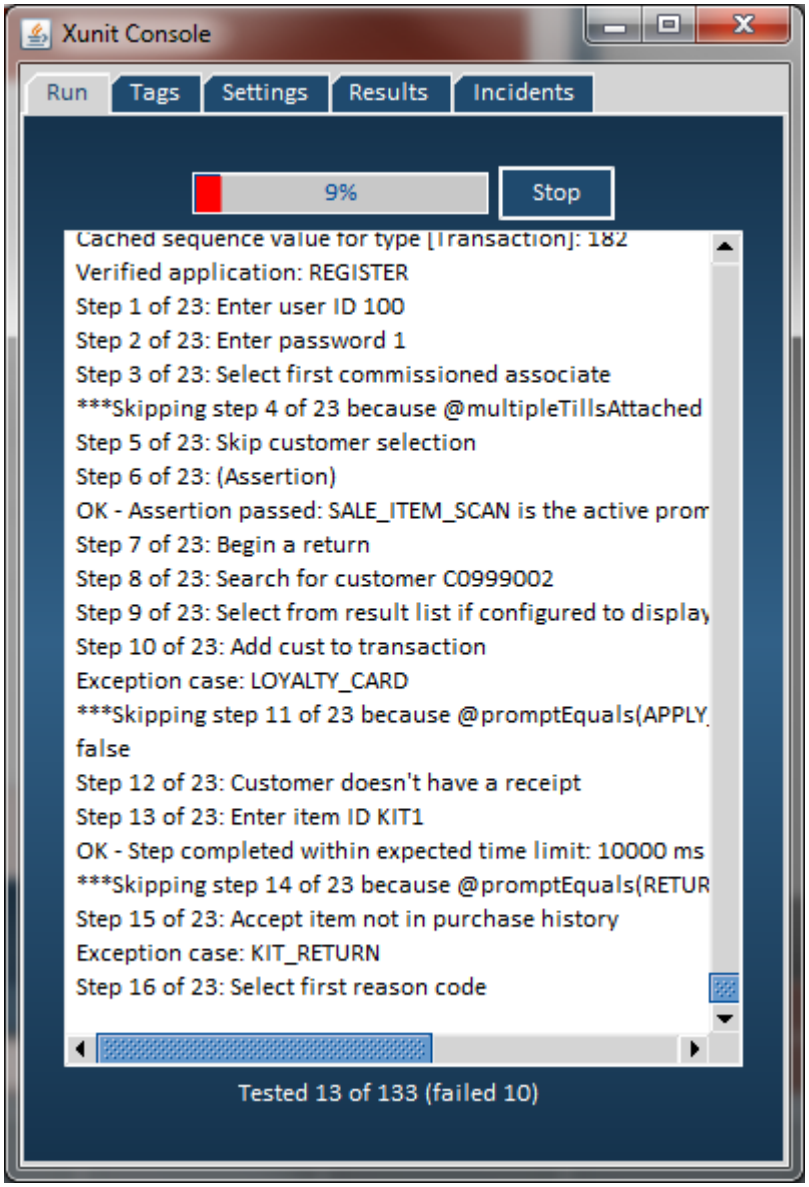
Running Test Cases

Click the **Start** button on the Xunit Console to run automated test cases on Xstore POS. Testing can be stopped by clicking the **Stop** button.

At the completion of the test cycle, Xunit displays a summary of the test results.

Copyright © 2025, Oracle and/or its affiliates.

Figure 4-2 Running Test Cases in Xunit



Messages

- Informational messages and test results are displayed on the Xunit Console in real time.
- When Xunit is configured to write messages to the xunit.out file (see log4j2–custom.xml in Enabling and Configuring Xunit), timestamps will be added to the messages in the xunit.out file.

Progress

- The progress bar displays the total progress of the entire configured test suite.
- The progress bar color indicates the status of the test suite:

Copyright © 2025, Oracle and/or its affiliates.

  - Blue (OK) - No test case failures have occurred and no incidents have been reported.
  - Yellow (Warning) - One or more incidents have occurred, but no test cases have failed. See Incidents.
  - Red (Error) - One or more test cases have failed. Note that incidents may also have occurred.
- A count of completed test cases, the total number of test cases, and the number of failed cases is displayed at the bottom of the Xunit Console screen.
- When test processing is complete, the **Start** button will be enabled and can be used to restart the test engine.

Failures

A test case fails when any of the following occur:

- The test step action to be performed at a specific processing point is not available to the user.
- A test step fails to execute within its configured time limit.
- An assertion on a test step evaluates to false.
- A “help desk” error is detected by Xunit.

When a test case fails, a system reset is performed, returning Xstore POS to the user login screen. The test suite will then continue with the next test case.

Incidents

Xstore POS may report incidents to Xunit at runtime. Incidents are issues that the tester should be made aware of, but don’t necessarily impact application or data flow through the system, or cause tests to fail.

- Incidents do not directly cause test cases to fail; their only purpose is to pass warnings along to the tester.
- Incidents are useful for detecting framework-level problems that would otherwise require assertions at every test step (for example, checking for missing translations).
- When Xunit receives an incident report from the system, the details are logged to the output console. An incident summary is also included with the Xunit test results when testing is completed.

> **Note**
>
> Incidents are created in Xstore POS code by calling the reportIncident method.

Selecting Tags

Test suites can be enabled through the Tags tab in the Xunit console. For more information about test suites, see TestSuite.

To enable test suites:

Copyright © 2025, Oracle and/or its affiliates.

1. Click the **Tags** tab in the Xunit console.
2. Click the **Edit** button.

   A list of available tags is displayed, with the number of associated test suites shown in parentheses next to the tag name.
3. If necessary, click the **Show suite names** check box to display the names of individual test suites.
4. Select the tags and/or test suites to test.
5. Click **Save**.

   The Xunit Tags window closes and the selected items are enabled for testing.

Xunit Setting

The Settings tab provides controls for Xunit behavioral settings.

Each time Xunit starts up, the settings in the Xunit console are reset to default values (configured in system.properties). Default values can also be restored by clicking the Reset button.

The Settings tab contains the following configurations:

- **Debug logging enabled** - Determines whether source logging will be performed for every test step. Source logging is useful for tracing the exact path of a test case.
- **Halt on failure** - Determines whether Xunit will halt immediately upon failing a test case.
- **Ignore test case iterations** - Determines whether Xunit will ignore any configured iteration values on RunTest elements during testing. Each test case will be executed only one time if this setting is selected.
- **Screen capture on failure** - If this setting is set selected, Xunit will save a screen capture to the export directory (see xunit.exportdir) when a test case fails.
- **Suppress receipts** - Determines whether receipts will print while test cases are running.
- **Step delay (ms)** - Sets the step delay (in milliseconds) when executing tests.
- **Max console lines** - Sets the number of log lines the console will display on the run tab.

HTML Error Report

Xunit produces a file named errorReport-{timestamp}.html with every test run that results in one or more test case failures. The file is written to the configured export directory (default location: C:\xunit-export). Open each file to view detailed information about the failed test case.

Copyright © 2025, Oracle and/or its affiliates.

5
Xunit Tips and Best Practices

This chapter describes best practices and recommendations when using Xunit.

Daily Testing

Developers working on customer projects using Xunit should run their default tag sets daily to ensure that the code base (and the test cases themselves) remain stable. Like anything in programming, letting something go for too long will eventually require large amounts of work to reconcile. By keeping tests cases up-to-date and running them regularly, you can ensure a high quality Xstore POS release with minimal feedback.

Ideally, test cases should be run automatically on a schedule as part of your environment. However, this setup doesn't preclude the need for developers to exercise the tests for any functional areas they happen to modifying prior to committing those changes to production code.

File Organization

Files should be organized consistently to ensure that testers and implementers can easily find and change configurations as necessary for testing. For this reason, you should follow certain conventions.

Base

Xunit configuration is based on directory structure. This means that Xunit configuration files can be named anything, but must reside in a configuration path subdirectory named xunit.

By convention, Xunit configuration files are named XunitConfig-<something>.xml. There are also a series of "meta" configuration files whose names have an underscore ("_") prefix. These are used to organize global elements, such as parameters, data pools, or assert types.

Customer

Customer projects should contain a version1/xunit directory and define Xunit configuration files in the following way:

- XunitConfig-overrides.xml - Used to override base configuration elements (test suites, test cases, sequences, parameters, data pools, and similar elements). Nothing else is overridden through this file.
- XunitConfig-common.xml - Used to define customer-specific configuration elements that are common to multiple activities or functional areas.
- XunitConfig-XXXXXX-descriptor.xml - Files with this naming format define configuration files for specific activities or functional areas, where:

Copyright © 2025, Oracle and/or its affiliates.

- XXXXXX is the activity number.
- descriptor is the brief descriptor for the functional area tested (for example, employeesale, pricememo, or coupon).

XML Schema

All Xunit configuration files should, in their headers, refer to the base XunitConfig.xsd schema file. This will ensure that configuration errors, misspellings and other similar errors will be found before XML files are deployed.

Performance Tips

Xunit will run to completion much more quickly if you do the following:

- Ensure you have a freshly built database.
- Disable calls to Oracle Retail Customer Engagement.
- Launch Xstore POS in Run mode rather than Debug mode.
- Ensure that DtvPreparedStatement logging is not set to Debug.
- Create a dummy printer on an unused port and set it to be your system's default printer.
- Limit the Eclipse console output:
  1. Open the **Preferences** menu in Eclipse.
  2. Click **Run/Debug**.
  3. Click **Limit console output**.
- Adjust memory for the Xstore POS launch configuration to 2GB (both min and max).
- Use the dtv.pos.ui.useFrame system property to minimize the Xstore POS window.
- Don not lock the computer's screen.

Copyright © 2025, Oracle and/or its affiliates.

6
Data Pools

This appendix lists the Data Pools available in Xunit.

Item

Item data pools retrieve data about items from the Xstore POS item database tables.

The item data pools can be organized into the following query keys:

- Standard-Use Items
- Item Dimensions
- Item UPC
- Attached Item
- Coupon Deal Items
- VAT Item
- Restricted Item
- Non-Merchandise Item

Standard-Use Items

Standard-use item query keys retrieve data from the Xstore POS item primary table itm_item.

Table 6-1 Standard-Use Item Data Pools

Name	Description
ITEMS	Standard-use item records. That is, items that do not prompt for quantity, price, and so on.
STYLE_ITEMS	Item styles.
NON_DISCOUNTABLE_ITEMS	Standard-use items that disallow discounts from being applied.
NON_DEAL_ITEMS	Standard-use items that are not eligible for deals.
WARRANTY_ITEMS	Standard-use item records that are warranty-eligible.
NON_INVENTORIED_ITEMS	Standard-use items that are not inventoried.
SERIALIZED_ITEMS	Standard-use items that required a serial number.
KIT_ITEMS	Standard-use kit items.
SERIALIZED_KIT_ITEMS	Standard-use items that belong to serialized kits.
ORDER_ITEMS	NA
PROMPT_FOR_QUANTITY_ITEMS	Standard-use items that prompt for quantity.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-1 (Cont.) Standard-Use Item Data Pools

Name	Description
PROMPT_FOR_DESCRIPTION_ITEMS	Standard-use items that prompt for description.
PROMPT_FOR_WEIGHT_ITEMS	Standard-use items that prompt for weight.
FORCE_QTY_OF_ONE_ITEMS	Standard-use items that can only have a quantity of one.
INITIAL_QTY_OF_TWO	Standard-use items that start with a quantity of two.
MIN_QTY_OF_TWO_ITEMS	Standard-use items that have to have at least a quantity of two.
MAX_QTY_OF_FOUR_ITEMS	Standard-use items that cannot have a quantity greater than four.
PROMPT_FOR_CUSTOMER_REQUIRED_ITEMS	Standard-use items that require a customer.
PROMPT_FOR_CUSTOMER_OPTIONAL_ITEMS	Standard-use items that prompt for, but do not require, a customer.
MIN_AGE_ITEMS	Standard-use items that require a minimum age of 21.
DISALLOW_CHANGE_ITEMS	Standard-use items that cannot be discounted or have the price changed.
DISALLOW_EXTENDED_TRANS_ITEMS	Standard-use items that cannot be used for extended transactions.
TAXABLE_ITEMS	Standard-use items that have a valid tax group and will be taxed.
NON_TAXABLE_ITEMS	Standard-use items that are not taxed.
PACK_ITEMS	Standard-use items that have a pack size.
XUNIT_LONG_SKU_ITEMS	Standard-use items that have long SKUs.
RESTOCKING_FEE_ITEMS	Standard-use items that include restocking fees.
ITEMS_WITH_UPC	Standard-use items with UPC codes.
ATTACHED_ITEMS	Items that are attached items.
ATTACHED_ITEMS_WITH_PROMPT	Attached items that prompt users to be attached.
XUNIT_COUPON_ITEMS	NA
XUNIT_VAT_ITEMS	NA
AGE_RESTRICTED_ITEMS	Standard-use items that have an age restriction.
ALWAYS_RESTRICTED_ITEMS	NA
NON_MERCH_ITEM_FEE	Non-merchandise fee items.
NON_MERCH_ITEM_FEE_PROMPT	Items that prompt for non-merchandise fees.

Table 6-2 Standard-Use Item Data Pool Fields

Name	Description
ItemId	Item ID.
Name	Item name.
Description	Item description.
PackSize	Item pack size.

Copyright © 2025, Oracle and/or its affiliates.

Item Dimensions

Item dimension query keys retrieve data from itm_item.

Table 6-3 Item Dimension Data Pool

Name	Description
DIMENSION_ITEMS	All items with dimensions configured.

Table 6-4 Item Dimension Data Pool Fields

Name	Description
ItemId	Item ID.
Dimension1	ID of first dimension.
Dimension2	ID of second dimension.
Dimension3	ID of third dimension.

Item UPC

Item UPC query keys retrieve data from itm_item_cross_reference.

Table 6-5 Item UPC Data Pools

Name	Description
ITEMS_WITH_UPC	All items with dimensions configured.

Table 6-6 Item UPC Data Pool Fields

Name	Description
Upc	Item UPC code.
ItemId	Item ID.
Manufacturer	Item manufacturer.
PrimaryFlag	Indicates whether the UPC is the primary UPC.

Attached Item

Attached item query keys retrieve data from itm_attached_items.

Table 6-7 Attached Item Data Pools

Name	Description
ATTACHED_ITEMS	All items with attached items and the items attached to them.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-7 (Cont.) Attached Item Data Pools

Name	Description
ATTACHED_ITEMS_WITH_PROMPT	Items that prompt whether to attach items, and the attached items.

Table 6-8 Attached Item Data Pool Fields

Name	Description
SoldItemId	ID of the item being sold.
AttachedItemId	ID of the item attached to the item being sold.
PromptToAdd	Indicates whether to prompt the user whether to attach the item.
QtyToAdd	Quantity of items to attach.

Coupon Deal Items

Coupon deal item query keys retrieve data from prc_deal_trig.

Table 6-9 Coupon Deal Item Data Pools

Name	Description
XUNIT_COUPON_ITEMS	Coupon items.

Table 6-10 Coupon Deal Item Data Pool Fields

Name	Description
DealId	ID of the deal.
ItemId	ID of the item.
CouponNbr	ID of the coupon.

VAT Item

VAT item query keys retrieve data from tax_tax_group_rule and itm_item_options.

Table 6-11 VAT Item Data Pools

Name	Description
XUNIT_VAT_ITEMS	Items with VAT taxes applied.

Table 6-12 VAT Item Data Pool Fields

Name	Description
ItemId	ID of the item.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-12 (Cont.) VAT Item Data Pool Fields

Name	Description
Name	NA
Description	NA
PackSize	Pack size of the item.

Restricted Item

Restricted item query keys retrieve data from itm_item_restriction.

Table 6-13 Restricted Item Data Pools

Name	Description
AGE_RESTRICTED_ITEMS	Items that are age-restricted.
ALWAYS_RESTRICTED_ITEMS	Items that are always restricted.

Table 6-14 Restricted Item Data Pool Fields

Name	Description
ItemId	ID of the item.
Name	Name of the item.
Description	Description of the item.

Non-Merchandise Item

Non-merchandise item query keys retrieve data from itm_item and itm_non_phys_item.

Table 6-15 Non-Merchandise Item Data Pools

Name	Description
NON_MERCH_ITEM_FEE	NA
NON_MERCH_ITEM_FEE_PROMPT	NA

Table 6-16 Non-Merchandise Item Data Pool Fields

Name	Description
ItemId	ID of the item.
Name	Name of the item.
Description	Description of the item.

Copyright © 2025, Oracle and/or its affiliates.

Customer

Customer data pools retrieve data from crm_party and other, related customer tables.

Table 6-17 Customer Data Pool

Name	Description
CUSTOMERS	All customer records.
CUSTOMERS_ELITE	All elite customers.
IN_COUNTRY_CUSTOMERS	Customers from a specific country.
TAX_EXEMPT_CUSTOMERS	Customers owning at least one active tax exemption in tax_tax_exemption.
NON_TAX_EXEMPT_CUSTOMERS	Customers without any active tax exemptions in tax_tax_exemptions.
HOUSE_ACCOUNT_CUSTOMERS	Customers owning at valid house account.
LAYAWAY_CUSTOMERS	Customers having at least one open layaway account in cat_cust_acct.
SP_ORDER_CUSTOMERS	Customers having at least one open special order account in cat_cust_acct.

Table 6-18 Customer Data Pool Fields

Name	Description
PartyId	Xstore POS party ID.
CustId	Xstore POS customer ID.
FirstName	First name.
LastName	Last name.
Address1	First line of the primary address.
City	City of primary address.
State	State or province of primary address.
Country	Country of primary address.
PostalCode	Postal code or zip code of primary address.

House Account

House Account data pools retrieve data from the cat_cust_acct table and related tables for Accounts Receivable users. Only accounts that exist at the current store are returned.

Table 6-19 House Account Data Pools

Name	Description
HOUSE_ACCOUNTS	All house accounts.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-19 (Cont.) House Account Data Pools

Name	Description
HOUSE_ACCOUNTS_WITH_BALANCE	House accounts with a balance.

Table 6-20 House Account Data Pool Fields

Name	Description
AccountCode	Account type.
CustId	ID of the customer associated with the account.
AccountId	Account ID.
AccountBalance	Balance on the account.
RetailLocationId	Retail location where the account originated.
IdentityReqFlag	Flag indicating whether a CUSTOMER typecode is required for the account type.
IdentityTypcode	Type of customer attached to the house account.
PartyId	Party ID to link to crm_party customer information.
AccountPoNumber	Purchase order associated with the account.
AccountStatCode	Status of account.
LastActivityDate	Date of last account activity.
AccountSetupDate	Date the account was created.
FirstName	First name on authorized account.
LastName	Last name on authorized account.
Telephone	Telephone number on account.
CreditLimit	Customer's credit limit.
PoRequredFlag	Whether a purchase order is required for using the account.
OnHoldFlag	Whether the account is on hold.
CorporateAccountFlag	Whether the account is a corporate account.

House Account New Users

House Account New Users data pools retrieve data about customers that have recently been added to house accounts.

Table 6-21 House Account New Users Data Pools

Name	Description
HOUSE_ACCOUNT_NEW_USERS	All new users on house accounts.
HOUSE_ACCOUNT_NON_PRIMARY_NEW_USERS	All new non-primary users on house accounts.
HOUSE_ACCOUNT_PRIMARY_NEW_USERS	All new primary users on house accounts.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-22 House Account New Users Data Pool Fields

Name	Description
AccountCode	Type of account.
AccountId	Account number in Xstore POS.
UserId	User ID for the authorized user on a payment account.
UserName	Displayed name of authorized user.
PartyId	Party ID of user.
EffectiveDate	Begin date for the user.
ExpirationDate	Date on which the authorization expires.
PrimaryContactFlag	Primary authorized user.
FirstName	User's first name.
LastName	User's last name.

House Account Users

House Account Users data pools retrieve data about customers associated with house accounts.

Table 6-23 House Account Users Data Pool

Name	Description
HOUSE_ACCOUNT_USERS	All users on house accounts.
HOUSE_ACCOUNT_NON_PRIMARY_USERS	All non-primary users on house accounts.
HOUSE_ACCOUNT_PRIMARY_USERS	All primary users on house accounts.
HOUSE_ACCOUNT_INACTIVE_USERS	All house account users in an inactive state.
HOUSE_ACCOUNT_NON_PRIMARY_ACTIVE_USERS	All non-primary users with an active status.

Table 6-24 House Account Users Data Pool Fields

Name	Description
AccountCode	Type of account.
AccountId	Account number in Xstore POS.
UserId	User ID for the authorized user on a payment account.
UserName	Displayed name of authorized user.
PartyId	Party ID of user.
EffectiveDate	Begin date for the user.
ExpirationDate	Date on which the authorization expires.
PrimaryContactFlag	Primary authorized user.
FirstName	User's first name.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-24 (Cont.) House Account Users Data Pool Fields

Name	Description
LastName	User's last name.

Transaction

Transaction data pools retrieve data from trn_trans and related tables. Only transactions for the current store and register are returned.

Table 6-25 Transaction Data Pools

Name	Description
COMPLETED_TRANS	Completed retail transactions.
CANCELLED_TRANS	Cancelled retail transactions.
VOIDED_TRANS	Voided retail transactions.
SUSPENDED_TRANS	Suspended retail transactions.

Table 6-26 Transaction Data Pool Fields

Name	Description
TransSeq	Sequence number.
BusinessDate	Business date.
WkstnId	Register ID.
RtlLocId	Store ID.
Total	Total amount.
TransTypeCode	Code identifying the type of transaction.
TransStatCode	Code identifying the status of the transaction.
PartyId	Party ID for the assigned customer.
CustomerId	ID for the customer.

Time Clock

Time clock data for employees. This data is retrieved from the thr_timeclk_trans table.

Table 6-27 Time Clock Data Pool

Name	Description
TIMECLOCK_TRANS	All time clock actions.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-28 Time Clock Data Pool Fields

Name	Description
TransSeq	Sequence number.
BusinessDate	Business date.
WkstnId	Workstation ID.
RtlLocId	Store ID.
PartyId	Party ID for the employee.
WorkCode	Work code for the time entry.
TimeclkEntryCode	CLOCK_IN or CLOCK_OUT

Customer Item Account

Information about customer item accounts, such as layaways, special orders, work orders, and send sales. Customer item account data pools retrieve data from cat_cust_acct and related tables.

Table 6-29 Customer Item Account

Name	Description
ALL_LAYAWAYS	All layaway accounts.
UNPAID_LAYAWAYS	Layaway accounts that have an unpaid balance.
CANCELABLE_LAYAWAYS	Open layaway accounts that are eligible to be canceled from the back office.
OPEN_LAYAWAYS	All open layaway accounts.
CLOSED_LAYAWAYS	All closed layaway accounts.
ALL_SPECIAL_ORDERS	All special order accounts.
OPEN_SPECIAL_ORDERS	Open special order accounts.
RECEIVED_SPECIAL_ORDERS	Special order accounts that have already been receiving but have not yet been completed.
CANCELABLE_SPECIAL_ORDERS	Special order accounts that are eligible to be canceled from the back office.
CLOSED_SPECIAL_ORDERS	Special order accounts that are closed.
ALL_SEND_SALES	All send sales that have been created.
ALL_WORK_ORDERS	All work order accounts.
OPEN_WORK_ORDERS	Open work order accounts.
RECEIVED_WORK_ORDERS	Work orders that are ready to be picked up.
CLOSED_WORK_ORDERS	Work orders that are closed.
ALL_PRESALES	All presale accounts.
OPEN_PRESALES	Open presale accounts.
RECEIVED_PRESALES	All presales that are ready to be picked up.
CLOSED_PRESALES	All presales that are closed.
ALL_ONHOLDS	All On Hold accounts.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-29 (Cont.) Customer Item Account

Name	Description
OPEN_ONHOLDS	Open On Hold Accounts.
RECEIVED_ONHOLDS	All On Holds that are ready to be picked up.
CLOSED_ONHOLDS	All On Hold accounts that are ready to be picked up.
ALL_LOCALORDERS	All local order accounts
OPENED_LOCALORDERS	All open local order accounts
READY_TO_PICKUP_LOCALORDERS	All ready to pickup loal order accounts
CLOSED_LOCALORDERS	All closed local order accounts

Table 6-30 Customer Item Account Data Pool Fields

Name	Description
CustAcctId	Customer item account ID.
LastName	Customer last name associated to the account.
FirstName	Customer first name associated to the account.
CustAcctCode	Code identifying the time of account (for example, layaway or special order).
CustAcctStatcode	Code identifying the account status.

Discount

Discount data pools provide information about discounts. The information is retrieved from the dsc_discount table and other, related tables.

Table 6-31 Discount Data Pools

Name	Description
LI_AMT_DISCOUNTS	Fixed-amount line item discounts.
LI_PCT_DISCOUNTS	Percentage line item discounts.
TRN_AMT_DISCOUNTS	Fixed-amount transaction discounts.
TRN_PCT_DISCOUNTS	Percentage transaction discounts.
GRP_AMT_DISCOUNTS	Fixed-amount group discounts.
GRP_PCT_DISCOUNTS	Percentage group discounts.
LI_EMPLOYEE_PCT_DISCOUNTS	Line item employee discounts.
TRN_EMPLOYEE_PCT_DISCOUNTS	Transaction employee discounts.
TRN_EXCLUSIVE_DISCOUNTS	All percent-off transaction discounts that are flagged as exclusive.
LI_AMT_DISCOUNTS_PROMPT	Amount prompt line item discounts
LI_PCT_DISCOUNTS_PROMPT	Percent prompt line item discounts

Copyright © 2025, Oracle and/or its affiliates.

Table 6-31 (Cont.) Discount Data Pools

Name	Description
TRN_PCT_DISCOUNTS_PROMPT	Percent prompt transaction discounts

Table 6-32 Discount Data Pool Fields

Name	Description
DiscountCode	Code identifying the discount.
CalcMethodCode	Xstore POS discount calculation method code.
Serialized	Indicates whether the discount requires a serial number to be entered.
Prompt	Indicates whether the associate will be prompted for the discount amount or percent to apply.

Inventory Document

Inventory document data pools return information about inventory documents. Data is retrieved from the inv_invctl_document table and other, related tables.

Table 6-33 Inventory Document Data Pools

Name	Description
OPEN_SEND_SALE_SHIPPING_DOCS	Open shipping documents for Send Sale transactions.
OPEN_RECEIVING_DOCS	Open receiving document line items.
OPEN_SO_RECEIVING_DOCS	Open special order receiving document line items.
IN_PROCESS_RECEIVING_DOCS	In-process document line items.
CLOSED_RECEIVING_DOCS	Closed receiving document line items.
IN_PROCESS_SHIPPING_DOCS	In process shipping document line items.
CLOSED_SHIPPING_DOCS	Closed shipping document line items.
INV_DOC_ADJ	Inventory adjustment document line items.
OPEN_INV_DOC_ADJ	Open inventory adjustment document line items.
OPEN_REPLENISHMENT_DOCS	Open replenishment document line items.

Table 6-34 Inventory Document Data Pool Fields

Name	Description
LocationId	Creating store ID.
DocumentId	ID for the document.
DocType	Type of document.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-34 (Cont.) Inventory Document Data Pool Fields

Name	Description
CartonId	ID for the carton on a document line item.
ItemId	ID for an item.
DocStatus	Status of the document.
BucketId	ID of the original inventory bucket for the line item.
DocSubType	Document subtype.

Inventory Count

Inventory count data pools provide information about inventory counts. Data is retrieved from the inv_count table and other, related tables.

Table 6-35 Inventory Count Data Pools

Name	Description
INV_COUNT	New (unprocessed) cycle count sheet line items.

Table 6-36 Inventory Count Data Pool Fields

Name	Description
CountId	ID of the inventory count.
CountTypeCode	Code identifying the type of inventory count.
BeginDate	Begin date for the count.
BucketId	ID of the inventory bucket for the line item.
ItemId	ID for the count sheet line item.

Inventory Valid Destination

Inventory valid destination data pools provide information about valid destinations for inventory. Data is retrieved from the inv_valid_destinations table.

Table 6-37 Inventory Valid Destination Data Pools

Name	Description
SHIPPING_DESTS_ST	Valid destinations for store transfer shipping documents.
SHIPPING_DESTS_DR	Valid destinations for defective return shipping documents.
SHIPPING_DESTS_RTV	Valid destinations for return to vendor shipping documents.
RECEIVING_DESTS_ASN	Valid destinations for ASN receiving documents.
RECEIVING_DESTS_ST	Valid destinations for store transfer receiving documents.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-38 Inventory Valid Destination Data Pool Fields

Name	Description
DocType	Inventory document type for which the destination is valid.
DocSubType	Document subtype for which the destination is valid.
DestinationId	ID for the destination. This is the store ID for store destinations and unique party identifier for non-store destinations.

Retail Location

Retail location data pools provide information about retail locations. Data is retrieved from the loc_rtl_loc table.

Table 6-39 Retail Location Data Pools

Name	Description
ALL_STORES	All defined retail locations, excluding the current location.
OTHER_COUNTRY_STORES	All retail locations in a country the current store is not in.
CURRENT_STORE	The current retail location.

Table 6-40 Retail Location Data Pool Fields

Name	Description
RtlLocId	Store ID.
City	City where the store is located.
State	State or province where the store is located.
PostalCode	Postal code or zip code for the store.
Country	Country where the store is located.

Tax Exemption

Tax exemption data pools provide information about tax exemptions. Data is retrieved from the tax_tax_exemption table.

Table 6-41 Tax Exemption Data Pools

Name	Description
TAX_EXEMPTIONS	All defined tax exemptions.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-42 Tax Exemption Data Pool Fields

Name	Description
TaxExemptionId	ID for the tax exemption.
PartyId	Party ID for the customer associated with the tax exemption.
CertificateNumber	Exemption certificate number.
ReasCode	Tax exemption reason code.
CertificateHolderName	Exemption certificate holder name.
ExpirationDate	Exemption expiration date.
CertificateState	State of the certificate.

Employee

Employee data pools provide information about employees. Data is retrieved from the hrs_employee table and other, related tables.

Table 6-43 Employee Data Pools

Name	Description
ACTIVE_EMPLOYEES	All currently active (that is, non-terminated) employees.
TERMINATED_EMPLOYEES	Terminated employees.
ACTIVE_EMPLOYEES_WITH_CUSTIDS	Currently active employees that have customer IDs.
OF_AGE_RESTRICTED_EMPLOYEES	Employees who are older than the restricted age.
UNDER_AGE_RESTRICTED_EMPLOYEES	Employees who are younger than the restricted age.
ALL_EMPLOYEES	All active employees, including the system employees.
POSTED_TIMECARD_EMPLOYEES	Active employees with posted timecard entries.
UNPOSTED_TIMECARD_EMPLOYEES	Active employees with not yet posted timecard entries.

Table 6-44 Employee Data Pool Fields

Name	Description
PartyId	Party ID for the employee.
FirstName	First name
LastName	Last name.
CustId	Employee's customer ID.
Address1	Primary street address line 1.
City	Primary address city.
State	Primary address state or province.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-44 (Cont.) Employee Data Pool Fields

Name	Description
Country	Primary address country.
PostalCode	Primary address postal code or zip code.
EmployeeId	Employee ID.
HireDate	Hire date.
ActiveDate	Date the employee was activated.
EmployeeStatusCode	Employment status.
JobTitle	Job title.
DepartmentId	Department ID.
EmployeeRoleCode	Role code.
EmployeeTypeCode	Type code.
EmployeeGroup	Employee group.
PersonalDays	Personal days remaining.
PersonalDaysUsed	Personal days used.
SickDays	Sick days remaining.
SickDaysUsed	Sick days used.
VacationDays	Vacation days remaining.
VacationDaysUsed	Vacation days used.
EmployeePayStatus	Pay status.
BasePay	Base pay rate.
AdditionalWithholdings	Additional tax withholdings.
OvertimeEligible	Indicates whether the employee is eligible for overtime.
ClockInNotRequired	Indicates whether the employee is required to clock in and out.
LastReviewDate	Last review date.
NextReviewDate	Next review date.
LoginId	Login ID.
PrimaryGroup	Primary group.
GroupMembership	Encoded group membership string.
TrainingStatus	Training status.
LockedOutFlag	Indicates whether the employee is currently locked out of the system.
TerminatedDate	Termination date.
BirthDate	Birth date.

Employee Borrow

Employee borrow data pools provide information about employee borrow functionality. Information is retrieved from hrs_employee.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-45 Employee Borrow Data Pools

Name	Description
EMPLOYEE_BORROW	Employees eligible to be borrowed by the current store.

Table 6-46 Employee Borrow Data Pool Fields

Name	Description
EmployeeId	Employee ID.
EndDate	Date after which the employee is eligible to be borrowed.

Employee Task

Employee task data pools provide information about employee tasks. Data is retrieved from the hrs_employee_task table.

Table 6-47 Employee Task Data Pools

Name	Description
ALL_EMPLOYEE_TASKS	All employee tasks.
OPEN_EMPLOYEE_TASKS	All employee tasks that have not been voided and are open.
OPEN_STORE_TASKS	All store tasks that have not been voided and are open.
OPEN_EMPLOYEE_GROUP_TASKS	All employee group tasks that have not been voided and are open.

Table 6-48 Employee Task Data Pool Fields

Name	Description
StartDate	Start date.
EndDate	End date.
TypeCode	Code identifying the type of task.
Visibility	Task visibility.
AssignmentId	ID of the assignment.
StoreCreatedFlag	Indicates whether the task was created at the store (rather than the home office).
Title	Task title.
Description	Description of the task.
Priority	Priority.
StatusCode	Code identifying the status.
TaskId	ID of the task.
VoidFlag	Indicates whether the task was voided.
PartyId	ID of the party assigned to the task.

Copyright © 2025, Oracle and/or its affiliates.

Employee Message

Employee message data pools provide information about employee messages. Data is retrieved from the hrs_employee_message table.

Table 6-49 Employee Message Data Pools

Name	Description
ALL_EMPLOYEE_MESSAGES	All employee messages.
NON_VOIDED_EMPLOYEE_MESSAGES	Employee messages that have not been voided.

Table 6-50 Employee Message Data Pool Fields

Name	Description
StartDate	Start date.
EndDate	End date.
Priority	Priority.
Content	Content of the message.
StoreCreatedFlag	Indicates whether the message was created at the store (rather than the home office).
WkstnSpecificFlag	Indicates whether the message is for a specific workstation.
WkstnId	ID of the register on which the message will be displayed.
MessageID	ID for the message.
VoidFlag	Indicates whether the message was voided.

Code Value

Code value data pools provide data about codes from different code categories. Data is retrieved from the com_code_value table.

Table 6-51 Code Value Data Pools

Name	Description
EMP_TASK_CODES	Code values for employee tasks.
EMP_PAY_STATUS_CODES	Code values for employee pay status.
EMP_WORK_STATUS_CODES	Code values for employee work status.
EMP_GROUP_CODES	Code values for employee groups.
EMP_ROLE_CODES	Code values for employee roles.
EMP_TYPE_CODES	Code values for employee types.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-52 Code Value Data Pool Fields

Name	Description
Name	Name of the code.
Description	Description of the code.

Reason Code

Reason code data pools provide information about reason codes for different reason code categories. Data is retrieved from the com_reason_code table.

Table 6-53 Reason Code Data Pools

Name	Description
CHANGE_FLOAT_REASONS	Reason codes for changing the register float amount.
DISCOUNT_REASONS	Reason codes for applying discounts to a transaction.
PAID_IN_REASONS	Reason codes for performing a paid in.
PAID_OUT_REASONS	Reason codes for performing a paid out.
POST_VOID_REASONS	Reason codes for post voiding a transaction.
PRICE_CHANGE_REASONS	Reason codes for performing a price override.
RETURN_REASONS	Reason codes for initiating a return.
TAX_CHANGE_REASONS	Reason codes for performing a tax override.
TAX_EXEMPT_REASONS	Reason codes for applying a tax exemption.
TILL_COUNT_REASONS	Reason codes for submitting a till count discrepancy.
TIME_OFF_REASONS	Reason codes for requesting time off.
TRANS_CANCEL_REASONS	Reason codes for canceling a register transaction.
VOID_LINE_REASONS	Reason codes for voiding a register transaction line item.

Table 6-54 Reason Code Data Pool Fields

Name	Description
ReasonCode	Reason code value.
Description	Description.
CommentReq	Indicates whether a comment is required.

Work Code

Work code data pools provide information about work codes. Data is retrieved from hrs_work_codes.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-55 Work Code Data Pools

Name	Description
WORK_CODES	Work codes.

Table 6-56 Work Code Data Pool Fields

Name	Description
WorkCode	Work code value.
Description	Description of the work code.

Security Group

Security group data pools provide information about security groups. Data is retrieved from the sec_groups table.

Table 6-57 Security Group Data Pools

Name	Description
SECURITY_GROUPS	All security groups.

Table 6-58 Security Group Data Pool Fields

Name	Description
GroupId	Security group ID.

Sales Goals

Sales goals data pools provide information about sales goals.

Table 6-59 Sales Goals Data Pools

Name	Description
SALES_GOALS	All sales goals.

Table 6-60 Sales Goals Data Pool Fields

Name	Description
SalesGoalId	ID of the sales goal.
Target	Sales goal target.
ToDate	How much of the sales goal has been hit already.

Copyright © 2025, Oracle and/or its affiliates.

Voucher

Voucher data pools provide information about vouchers. Data is retrieved from the ttr_voucher_history table.

Table 6-61 Voucher Data Pools

Name	Description
GIFT_CERTIFICATE_ACTIVITIES	Any gift certificate tender account activities.
STORE_CREDIT_ACTIVITIES	Any store credit tender activities.

Table 6-62 Voucher Data Pool Fields

Name	Description
TransSeq	Sequence number of the sales transaction.
BusinessDate	Business date of the sales transaction.
WkstnId	ID of the workstation.
RtlLocId	ID of the store.
VoucherTypeCode	Type of voucher.
SerialNumber	Serial number of the voucher.
ActivityCode	The type of activity on the account.
Amount	Amount added to or subtracted from the voucher.

Raincheck

Raincheck data pools provide information about rain checks. Data is retrieved from the trn_raincheck table.

Table 6-63 Raincheck Data Pools

Name	Description
RAIN_CHECKS	All rain checks that have been entered.
UNREDEEMED_RAIN_CHECKS	All rain checks that have been entered, but not redeemed.

Table 6-64 Raincheck Data Pool Fields

Name	Description
RainCheckId	ID for the rain check.
ItemId	ID of the item on the rain check.
SalePrice	Price of the item on the rain check.
ExpirationBusinessDate	Rain check expiration date.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-64 (Cont.) Raincheck Data Pool Fields

Name	Description
RedeemedFlag	Indicates whether the rain check was redeemed.

Customer Entry

Customer entry data pools provide test data that can be used to create new customers in Xstore POS, or otherwise populate Xstore POS data entry forms. Data for these pools is retrieved from flat text files, rather than the Xstore POS database.

Table 6-65 Customer Entry Data Pools

Name	Description
ENTER_CUSTOMER	Customer test data.

Table 6-66 Customer Entry Data Pool Fields

Name	Description
Salutation	The customer's salutation (Mr., Mrs, Ms., Dr., and so on).
FirstName	First name.
LastName	Last name.
Address1	First line of the address.
Address2	Second line of the address.
City	City of address.
State	State or province of address.
Country	Country of address.
EmailAddress	Email address.
Telephone1	Primary telephone number.
Telephone2	Secondary telephone number.
EmailContact	Indicates whether the email address should be used to contact the customer.
Telephone1Contact	Indicates whether the primary telephone number should be used to contact the customer.
Telephone2Contact	Indicates whether the secondary telephone number should be used to contact the customer.
ContactPref	The customer's preferred contact method.

Shift

Shift data pools hold all employee work shift information. Data is retrieved from the sch_shift table.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-67 Shift Data Pools

Name	Description
SHIFTS	All employee shifts.
VOIDED_SHIFTS	All employee shifts that have been voided.

Table 6-68 Shift Data Pool Fields

Name	Description
ShiftId	ID of the shift.
WorkCode	Work code assigned to the shift.
Name	Name of the shift.
Description	Description of the shift.

Time Off

Time off data pools provide information about employee time off. Data is retrieved from the sch_emp_time_off table.

Table 6-69 Time Off Data Pools

Name	Description
TIMEOFF	All employee time off information.

Table 6-70 Time Off Data Pool Fields

Name	Description
EmployeeId	ID of the employee.
TimeOffId	ID of the time off request.
ReasonCode	Reason code for the time off.

Stock Ledger

Stock ledger data pools provide information about stock ledgers. Data is retrieved from the inv_stock_ledger_acct table.

Table 6-71 Stock Ledger Data Pools

Name	Description
ON_HAND_AT_VENDOR_STOCK	All items with 10 quantity in the ON_HAND bucket, and less than that in the AT_VENDOR bucket.
ON_HAND_LAYAWAY_STOCK	All items with 10 quantity in the ON_HAND bucket, and less than that in the LAYAWAY bucket.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-71 (Cont.) Stock Ledger Data Pools

Name	Description
ON_HAND_ONHOLD_STOCK	All items with 10 quantity in the ON_HAND bucket, and less than that in the ONHOLD bucket.
ON_HAND_ORDER_STOCK	All items with 10 quantity in the ON_HAND bucket, and less than that in the ORDER bucket.
ON_HAND_PRESALE_STOCK	All items with 10 quantity in the ON_HAND bucket, and less than that in the PRESALE bucket.
ON_HAND_SPECIAL_ORDER_STOCK	All items with 10 quantity in the ON_HAND bucket, and less than that in the SPECIAL_ORDER bucket.
ON_HAND_WORK_ORDER_STOCK	All items with 10 quantity in the ON_HAND bucket, and less than that in the WORK_ORDER bucket.

Table 6-72 Stock Ledger Data Pool Fields

Name	Description
ItemId	ID of the item.

Serialized Stock Ledger

Serialized stock ledger data pools provide information about ledger information. Data is retrieved from the inv_stock_ledger_account table.

Table 6-73 Serialized Stock Ledger Pools

Name	Description
LAYAWAY_STOCK	Items in the LAYAWAY inventory bucket.
ON_HAND_SERIALIZED_STOCK	Serialized Items and their current ON_HAND count.

Table 6-74 Serialized Stock Ledger Data Pool Fields

Name	Description
ItemId	Item ID.
BucketId	ID of the inventory bucket.
Unitcount	Stock count for the item in the specified inventory bucket.
SerialNumber	Item serial number.

Sold Serial Numbers

Sold serial numbers data pools provide serial number information for items that have already been sold. Data is retrieved from the inv_inventory_journal table.

Copyright © 2025, Oracle and/or its affiliates.

Table 6-75 Sold Serial Number Pools

Name	Description
RETURN_SERIAL_NUMBERS	Serial numbers of returned items.

Table 6-76 Sold Serial Number Pool Fields

Name	Description
TransSeq	Transaction sequence number.
TransLineItemSeq	Sequence number of the line item within the transaction.
ItemId	ID of the item.
SerialNbr	Serial number.

Order

Order data pools provide information about orders from the Order Broker Cloud Service. Data is retrieved from the xom_order table.

Table 6-77 Order Data Pools

Name	Description
ALL_ORDERS	All Order Broker Cloud Service orders.
OPEN_DELIVERY_ORDERS	All Order Broker Cloud Service delivery orders in the open status.
OPEN_OS_PICKUP_ORDERS	All Order Broker Cloud Service pickup at other store orders in the open status.
OPEN_TS_PICKUP_ORDERS	All Order Broker Cloud Service pickup at this store orders in the open status.
REJECTED_ORDERS	All Order Broker Cloud Service orders in the rejected status.
READY_FOR_PICKUP_OS_PICKUP_ORDERS	All Order Broker Cloud Service pickup from other store orders in the ready for pickup status.

Table 6-78 Order Data Pool Fields

Name	Description
OrderId	ID of the order.
OrderType	Order type.
StatusCode	Status of the order.
ShipComplete	Indicates whether the shipment is complete.
UnderReview	Indicates whether the order need to be reviewed.
LineStatusCode	Status code of an individual line item.

Copyright © 2025, Oracle and/or its affiliates.

7
Evaluation Methods

Evaluation methods retrieve system information from Xstore POS.

> **Note**
>
> For information and examples on how to use evaluation methods, see @ Evaluation Method Chain in Chapter 3, Creating and Configuring Test Cases.

Table 7-1 Evaluation Methods

Method	Description
BigDecimal add((BigDecimal argAddend1, BigDecimal argAddend2)	Arithmetic addition. Returns a BigDecimal sum of the argument values.
boolean and(boolean argCondition1, boolean argCondition2)	Logical AND. Returns a boolean indicating whether both conditions are met.
Date clearTime(Date argDate)	Clears time from the provided date. Returns a Date with the new date value.
boolean equals(Object arg01, Object arg02)	Determines whether arg01 equals arg02 are equal. Returns a boolean indicating whether the arguments contain equal values.
boolean equalsIgnoreCase(String argS1, String argS2)	Determines whether arg01 equals arg02 when letter case is ignored. Returns a boolean indicating whether the arguments contain equivalent strings when ignoring letter case.
String generateCardNumber(String argType)	Generates a random, valid card number for the specified input type from InputConfig.xml. Returns a String containing the card number.
BigDecimal getActualPrice(String argItemId)	Retrieves the price of the specified item. Returns a BigDecimal with the price of the item.
BigDecimal getActualPrice(String argItemId, BigDecimal argQty)	Retrieves the price of the specified item with the specified quantity. Returns a BigDecimal with the price of the specified quantity of the item.
BigDecimal getCashAmountDue()	Gets the cash amount due for the current transaction. Returns a BigDecimal with the amount due in the local currency.
String getBarcode(PosTransactionId argTransactionId)	Gets the barcode string for the specified POS transaction ID. Returns a String with the barcode for the POS transaction.

Copyright © 2025, Oracle and/or its affiliates.

Table 7-1 (Cont.) Evaluation Methods

Method	Description
BigDecimal getBigDecimalValue(Object argValue)	Converts string to decimal to perform math functions and Assertions. This includes the following submethods: • subtract • negate • add Returns a BigDecimal value for the string.
BigDecimal getBucketCount(String argItemId, String argBucketId)	Retrieves the count of a specified item in a specified inventory bucket in the local store. Returns a BigDecimal count of the item.
Date getBusinessDateTimeStamp()	Returns a Date with the time stamp for the current business date.
Date getCurrentBusinessDate()	Returns a Date with the time stamp for the current business date.
String getCurrentContext()	Returns a String with the current context.
ICustomerAccount getCurrentCustomerAccount(String argAccountType)	Returns the account information for a particular type of customer account. This includes the following submethods: • getCustAccountId • getAccountBalance • getDeliveryModifier • getDeliveryType • getCustAccountCode • getAccountPurchaseOrderNumber
String getCurrentEmployeeId()	Returns a String containing the employee ID for the currently logged-in employee.
IPosTransaction getCurrentOriginalTransaction()	Retrieves the original transaction associated with the current transaction. Has the following submethods: • getTransactionStatusCode • getTransactionSequence • getSubtotal • getTaxAmount • getRoundedAmount • getTotal • getTransactionTypeCode
CountSummaryItem getCurrentTillCountItem()	Returns a CountSummaryItem object containing the current count summary item being counted.
String getCurrentTillCountTenderType()	Returns a String with the name of the current tender type being counted.
long getCurrentTime()	Returns a long with the current business date timestamp in milliseconds.
IParty getCustomerParty()	Returns an IPart object containing party information for the current customer.
int getDataPoolSize(String argPool)	Used to support DataPool dump into Console. Returns an int.
String getDefaultShippingMethod()	Returns String with the default shipping method.

Copyright © 2025, Oracle and/or its affiliates.

Table 7-1 (Cont.) Evaluation Methods

Method	Description
String getEmptyString()	Returns an empty String. This method allows blank values to be passed as sequence arguments.
long getFileSize(String argFilePath)	Retrieves the size of the specified file in bytes. Returns a long with the size of the specified file in bytes.
BigDecimal getForeignCurrencyAmount(BigDecimal argLocalCurrencyAmount, String argCurrencyId)	Converts a specified amount of local currency to its value in a specified foreign currency. Returns a BigDecimal with the foreign currency value.
List<String> getInvDocSubTypesForSearch(String argType)	Gets a list of valid inventory document subtypes for search. Returns a List<> of String values, each of which is a valid inventory document subtype for search.
BigDecimal getLastClosingCashAmt(String argCurrency)	Retrieves the amount of currency in the last store closing. Returns a BigDecimal with the amount of the currency.
BigDecimal getLastStoreSafeCashAmt(String argCurrency)	Retrieves the amount of currency in the last store closing. Returns a BigDecimal with the amount of the currency.
IPosTransaction getLastTransaction()	Returns information in memory for the prior transaction. Has the following submethods: • getTransactionStatusCode • getTransactionSequence • getSubtotal • getTaxAmount • getRoundedAmount • getTotal • getTransactionTypeCode
BigDecimal getLastTransactionForeignTotal()	Returns a BigDecimal with the total amount of foreign currency.
PosTransactionId getLastTransactionId()	Returns information about the prior transaction. Includes the following submethods: • getRetailLocationId • getBusinessDate • getWorkstationId • getTransactionSequence
BigDecimal getLastTransactionTotal()	Returns a BigDecimal with the value of the prior transaction total.
ICombinedListModel<?> getListModel(String argModelKey)	Helper methods to select information in Inventory screens. Includes the following submethods: • getSelectedElements • get(indexValue) • getCount • getShelfPrice • getStockLabel • size
int getListModelSize()	

Copyright © 2025, Oracle and/or its affiliates.

Table 7-1 (Cont.) Evaluation Methods

Method	Description
int getListModelSize(String argModelKey)	
long getLogFileSize(String argLogType)	Returns a long value of the size of the specified log file in bytes.
String getNextSequenceValue(String argSequenceType)	Returns the next sequence number for the specified sequence type. Returns a String containing the next sequence number.
ISaleReturnLineItem getOriginalSaleLine(int argIndex)	Retrieves the sale line from the original transaction associated with the current transaction.
IDtvDateRange getRelativeDateRange(String argName)	
IRetailLocation getRetailLocation()	Returns information about the current location.
int getRetailLocationId()	Returns an int value of the ID for the retail location.
String getRetailLocationState()	Returns a String indicating whether the local store is open or closed.
BigDecimal getRoundedTotal()	Returns a BigDecimal value of the rounded total of the transaction.
BigDecimal getRoundedTotal(BigDecimal argTotal)	Calculates what the rounded amount would be for a given amount. Returns a BigDecimal value fo the rounded value for the argument.
ISaleReturnLineItem getSaleLine(int argIndex)	Retrieves a given sale line item. This has the following submethods: • getItemId • getBaseUnitPrice
ISaleReturnLineItem getSaleLine(IPosTransaction argTrans, int argIndex)	Retrieves a given sale line item. This has the following submethods: • getItemId • getBaseUnitPrice
BigDecimal getSessionTenderAmount(String argTenderId)	Returns a BigDecimal value of the current session tender amount for a given tender type.
Date getSystemTime()	Returns a Date with the current system time.
ITender getTender(String argTenderId)	Returns information about a particular tender. Includes the following submethods: • getOptions • getCidKeyedRequired • getCustIdReqCode • getSerialIdentificationNbrRequired

Copyright © 2025, Oracle and/or its affiliates.

Table 7-1 (Cont.) Evaluation Methods

Method	Description
IPosTransaction getTransaction()	Returns information about the current Transaction. Includes the following submethods: • getTotal • getTotal.abs • getTaxAmount • getAmountDue • getAmountDue.abs • getTransactionTypeCode • getTransactionSequence • getKeyedOffline • getSubTotal • getRoundedAmount • getTransactionStatusCode
int getWorkstationId()	Returns an int value with the ID of the workstation.
String getWorkstationState()	Returns a String indicating whether the workstation is open or closed.
Date getXDaysFromDate(int argDays, Date argDate)	Calculates the date which is the specified number of days from the provided date. Returns a Date with the calculated value.
Date getXDaysFromNow(int argDays)	Calculates the number of days after the current business date. Returns a Date with the calculated value.
Date getXHoursFromDate(int argHours, Date argDate)	Calculates the date which is the specified number of hours from the provided date. Returns a Date with the calculated value.
Date getXHoursFromNow(int argHours)	Calculates the date which is the specified number of hours from the current business date timestamp. Returns a Date with the calculated value.
boolean invert(boolean argBool)	Returns a boolean that is the inversion of the argument.
Date getTimeOfDay(int argHour, int argMinute)	Gets a date object for the current business date at the specified hour and minute. Returns a Date with the specified hours and minutes.
Object invokeConfigMgr(String argMethodName)	Invokes the specified static base configuration manager method and returns the result.
Object invokeStatic(String argMethodName, String argClassName)	Invokes a static method on a class.
boolean isClosingMessageAvailable()	Returns a boolean indicating whether there is a closing message configured.
boolean isLayawayDepositRequired()	Returns a boolean indicating whether a layaway deposit is required.
boolean isNonNull(Object argO)	Returns a boolean indicating whether an object is not null.
boolean isPositive(BigDecimal argNumber)	Returns a boolean indicating whether the given number is positive.
boolean isReasonCommentRequired(String argValue)	Returns a boolean indicating whether the value provided for the reason code comment flag should result in prompting for comment.

Copyright © 2025, Oracle and/or its affiliates.

Table 7-1 (Cont.) Evaluation Methods

Method	Description
boolean isTrainingMode()	Returns a boolean indicating whether the register is in training mode.
boolean multipleTillsAttached()	Returns a boolean indicating whether multiple tills are attached to this workstation.
boolean isSerialNumberOnFile(String argItemId, String argSerialNumber, String argBucketId)	Checks if the specified serial number exists in the serialized stock ledger table for the current store. Returns a Boolean indicating if the serial number was found.
BigDecimal negate(BigDecimal argNumber)	Returns a BigDecimal value that is the negative of the argument value.
boolean or(boolean argCondition1, boolean argCondition2)	Logical OR. Returns a boolean indicating whether either condition is met.
String peekNextSequenceValue(String argSequenceType)	Returns a String value that is the next value in a sequence.
boolean promptEquals(String argPromptKey)	Returns a boolean value indicating whether the currently active prompt or form is associated with the specified prompt key.
BigDecimal subtract(BigDecimal argSubtrahend1, BigDecimal argSubtrahend2)	Arithmetic subtraction. Returns a BigDecimal difference of the argument values.
String translate(String argTranslationKey)	Retrieves the text for the translation key for the configured language. Returns a String containing the text for the specified translation key.
Object checkSysConfig(String argSettingName)	Used to the check of the SysConfig.xml setting that is specified by argSettingName. Returns the value of the setting.

Copyright © 2025, Oracle and/or its affiliates.

8
Global Exceptions

This appendix contains a list of prompts that can be handled by exceptions, and under what circumstances they appear.

Table 8-1 Global Exceptions

Exception	Description
APPLY_TRANS_TAX_EXEMPTION	No transaction tax exemptions are applied for customers with tax exempt certificate on file.
BELOW_MIN_CASH_AMOUNT	If the minimum cash in the drawer is below the configurable amount, this message appears. Automatically cleared.
CLOSE_CASHDRAWER_CHANGE_OK	If the cash drawer need to be closed after change is given.
CREDIT_VOUCHER_SIGNATURE	Prompt for a signature on a credit voucher.
DISCOUNT_AUTO_REMOVE	If there is a warning when you have a discount removed/replaced, it is accepted automatically.
DISCOUNT_PROMPT	An informational prompt about the chosen discount.
EMAIL_RECEIPTS	Default for all transactions is to print receipts. This prompts at the end of the transaction, after tendering.
EXCEED_MAX_CASH_AMOUNT	Maximum cash in drawer threshold is reached, automatically cleared.
EXCEED_MAX_TILL_AMOUNT	Maximum currency amount is in the till.
FORCE_CCA_PICKUP_ALL	Handles the 'Pickup all items' message for a CCA transaction.
FORCE_PICKUP_ALL	Handles the 'Pickup all items' message for any layaway transactions.
IMPRINT_CREDIT_CARD	Message to imprint card after manual card entry is cleared automatically.
JOIN_LOYALTY_CUSTOMER_CENTRIC	Prompt for a customer to join the loyalty program.
KIT_RETURN	An informational prompt when a KIT item is returned.
LAYAWAY_DEPOSIT_OVERRIDE	If the layaway amount for deposit is over the configured amount, this message displays. Exception clears the message with 'YES' action.
LOYALTY_CARD	A prompt at the beginning of a transaction asking to add a loyalty card number, or sign the customer up for a loyalty card.
NO_CHALLENGE_QUESTIONS	A prompt that the current employee does not have an challenge questions selected.
NO_SIGNATURE_REQUIRED	A prompt that no signature is required.
OLD_INVENTORY_DOCUMENTS	Any 30 day old inventory docs are displayed on a prompt before you can search. This continues past that list to the search screen.

Copyright © 2025, Oracle and/or its affiliates.

Table 8-1 (Cont.) Global Exceptions

ORDER_CONFIRM_ON_HAND_ITEM	A prompt that shows when trying to place an Order Broker order but there is stock on hand.
PAYROLL_ERROR_LIST	An error message when not all employee's payroll has been reviewed.
POST_VOID_TRANSACTION_LIST	List of transactions for a post void.
PRINT_RECEIPTS	If the prompt displays that shows print and email as an option, this one is chosen.
PRINTER_ERROR	Prompt indicating an error with the printer.
PROMPT_FOR_ROUNDUP_DONATION	Default to not donate change to charity.
PROMPT_LOYALTY_CARD	Prompt for loyalty card is passed.
PROMPT_VOID_DONATION_ITEM	A prompt when a donation line item was selected to be deleted.
RECEIVED_SP_ORDER_ACCOUNT_LIST	List of 'Ready to Pickup' Special Orders will display before any Special Order search actions are available. This exception clears the list.
RECOMMENDED_LAYAWAY_PAYMENT_CHECK	Message prompt for checking the recommended layaway amount.
RESTOCKING_FEE_APPLY_MESSAGE	A message that a restocking fee will be allied to the return.
RETURN_PRICE_HISTORY_EMPTY	Accepts the message that shows that there is no return price history.
SALE_COMPLETE_CHECK	"Is this sale complete?" message handled automatically.
SCHEDULE_TIME_OFF_NOTIFY	A prompt that appears when time off is scheduled when the employee is already scheduled to work.
SIGNATURE_REQUIRED	When there is a signature required on the receipt this handles the message.
SP_ORDER_CONFIRM_ONHAND	Confirmation that you still want to Special Order an item that has an on hand inventory.
SUSPEND_TRANSACTIONS_TO_CANCEL	Prompt when the store is being closed while suspended transactions still exist.
WORK_ORDER_REQ_ATTENTION_LIST	Popup ListPrompt with work orders that can be worked.
PRINT_ACKNOWLEDGEMENT	Prompt for printing a tax free document.

Copyright © 2025, Oracle and/or its affiliates.